ViCSI

Complete Engineering and Maintenance Manual

A from-source engineering reference for the ViCSI VR crime-scene-investigation training simulator (Unity 6000.5.6f1, Meta Quest) and its vicsi-dashboard backend (Next.js + Supabase).

Snapshot date: 2026-09-19 • Branch: master • HEAD: 0ab53f2

Built by direct repository inspection. Unverified claims are marked explicitly throughout.

Table of Contents

ViCSI — Complete Engineering and Maintenance Manual

Quick Start for a New Developer

Table of Contents

1. Current Project Snapshot

2. Repository Structure

2.1 Top-level layout

2.2 `Assets/_Project/` — the actual project

2.3 Packages and project configuration

2.4 `Tools/Audit/` — Node-based read-only audit tooling

2.5 Prior documentation — read this before trusting anything else in `docs/`

3. GameObject / Scene Architecture

3.1 `_Managers` — the single most important GameObject in the project

3.2 Other root-level GameObjects (selected — full list of 46 roots captured; only gameplay-relevant ones detailed here)

3.3 Common component signature for pickupable evidence

3.4 Cross-reference: what to do if you need to see the FULL live hierarchy yourself

4. Script Catalog — CaseFile, RoleSystem, ProceduralGate, Core

4.1 CaseFile folder (11 files) — evidence identity, state, and custody

4.2 RoleSystem folder (9 files) — tool equip machinery

4.3 ProceduralGate folder (1 file)

5. Script Catalog — Requirements (the shared evaluator)

6. Script Catalog — Interaction (all tools; 34 files)

6.1 The generic two-handed equip pattern

6.2 Catalog (all 34 files)

7. Script Catalog — UI, Feedback, and VR Input Architecture

7.1 `PlayerUIGate.cs` — the UI mutual-exclusion primitive

7.2 `HudFollowCamera.cs` — the world-space-canvas-follows-HMD pattern

7.3 UI file catalog (`Assets/_Project/Scripts/UI/`, 12 files)

7.4 `Assets/_Project/Scripts/Feedback/` (2 files)

7.5 Consolidated controller-input table

7.6 How to add a new controller action

8. VR / Meta XR Architecture

9. Data Model — ScriptableObjects and Data/ Assets

9.1 Every ScriptableObject type (10 found via whole-tree grep for `: ScriptableObject`)

9.2 `Data/` folder inventory (by actual verified type, not folder name)

9.3 A concrete worked example — `EVD-014_KitchenKnife.asset` (raw YAML excerpt)

9.4 Enum order-sensitivity audit — full list

10. Evidence System — full lifecycle (verified against source, not assumed)

11. Evidence Inventory / Held-Item System (new subsystem — read this before touching custody/equip code)

12. Requirement System — how to add a new requirement

13. Deduction / Story / Ending System
- 13.1 The chain, end to end
- 13.2 Component-by-component
- 13.3 A second concrete worked example — the highest-priority ending
- 13.4 Corkboard / Deduction Board status — dormant, not broken

14. Complete Game Flow (chronological, script-level)

15. Logging / Assessment Pipeline (full detail)
- 15.1 `SessionLogger` and the local event log
- 15.2 Building the assessment payload
- 15.3 Upload service — queue, retry, and the restart-authentication design
- 15.4 `AssessmentUploadConfiguration` — the fail-closed gate, and current risk

16. Backend Boundary (Unity ↔ vicsi-dashboard ↔ Supabase)
- 16.1 Pairing: `POST /api/v1/device-pairings/exchange`
- 16.2 Session upload: `POST /api/v1/assessment-sessions`
- 16.3 Schema / RLS summary (16 migrations, in order)

17. Editing Without Unity UI

18. Debugging Without Unity UI

19. Testing

20. Git / Change Management

21. "I Need to Change X → Go Here" Quick Reference

22. Safe Modification Matrix

23. Dependency Diagrams
- 23.1 High-level architecture
- 23.2 Evidence lifecycle
- 23.3 Requirement evaluation flow
- 23.4 Deduction / ending flow
- 23.5 Session logging → assessment upload flow
- 23.6 Input → Tool/UI flow

24. Glossary

25. Beginner Learning Path

26. Documentation Quality Self-Check

27. Appendix — This Session's Changes (for context, not exhaustive project history)

ViCSI — Complete Engineering and Maintenance Manual

This manual was built by directly inspecting the repository at `C:\Users\Roduf Eleu\My project (1)` (Unity project) and `C:\Users\Roduf Eleu\Desktop\vicsi-dashboard` (backend) on 2026-09-19. It is a snapshot, not a promise — the working tree had 59 changed/untracked paths at the time of writing (see Current Project Snapshot). Facts are marked VERIFIED (read directly from source this pass), INFERRED (reconstructed from related code/comments but not independently opened), or NOT VERIFIED (could not be confirmed from the repository). Nothing in this document was invented; where the repository did not answer a question, that is stated explicitly rather than guessed.

Quick Start for a New Developer

1. **This is a Unity 6000.5.6f1 VR training simulator** for Meta Quest — a player investigates a simulated crime scene (`CSI_Environment` scene), collects and documents evidence through a fixed procedural sequence, reasons about relationships between evidence, answers hypothesis checkpoints, and submits a conclusion that resolves to one of five authored endings. A companion Next.js + Supabase web app (`vicsi-dashboard`, separate repo) handles student/instructor accounts, pairs a headset to a real student identity, and receives an uploaded session for scoring.
2. **Almost the entire game-state layer lives on one GameObject**, `_Managers`, as ~25 sibling `MonoBehaviour` components (`EvidenceStateManager`, `SessionLogger`, `ObjectiveStateManager`, `EndingManager`, `AssessmentUploadService`, etc.). This is the single most important structural fact in the scene — see Section 4.
3. **Read** `EvidenceStateManager.cs` **first.** Nearly every other system either writes to it or reads from it. Its `RequiredSequence` array is the canonical evidence lifecycle order; its `EvidenceStatus` enum values are persisted in `.asset` YAML and must never be reordered.
4. **The codebase has a strong, explicit house style you should follow**: enums that are ever persisted (into `.asset` YAML or a JSON log) are given explicit integer values and are documented as append-only-never-reorder. This has already caused two real, cited incidents (`EvidenceStatus`, `ToolType`) and the project's own comments show the scars.
5. **There is no separate character/role system.** One player, many equippable `PlayerTool`s (camera, swab, fingerprint kit, evidence bag, recorder, chalk, tent dispensers, flashlight...), selected from a radial "tool wheel." `ToolType` just tags which capability is currently equipped.
6. **UI is not Unity's normal in-editor uGUI event pipeline.** Every panel in this project uses raw `InputAction` polling (thumbstick + face buttons) instead of pointer/ray-based selection, because the project's own testing confirmed the Meta ISDK ray→uGUI pipeline never worked reliably against these runtime-built World Space canvases. If you add new UI, follow this pattern, not Unity's default one.
7. **The Assessment/pairing/upload subsystem and the Evidence Inventory/Custody/HeldItem subsystem are both brand new and entirely uncommitted** as of this snapshot. Five pre-existing documentation files in `docs/` and the repo-root `VR-CSI_SESSION_HANDOFF.md` predate both and say nothing about them — do not trust those older docs on evidence custody, inventory, pairing, or upload topics; this manual supersedes them there.
8. Before touching anything, run `git status` and read Section 18 (Git / Change Management) — there is a large amount of real, valuable, uncommitted work sitting in the working tree right now.

Table of Contents

(Generated automatically in the PDF from the headings below.)

1. Current Project Snapshot

Branch: `master`. **HEAD**: `0ab53f20e23c91a14c4725487e5b5d4ec881f6b1` — "Add ViCSI requirement/relationship/narrative/objective/ending systems and audit tooling" (2026-09-15).

Working tree: NOT clean. 59 changed/untracked paths. 27 tracked files modified (+7322/-2690 lines, `git diff --stat`), including a 9,089-line diff on `Assets/_Project/Scenes/Scenarios/CSI_Environment.unity` alone. A full untracked subsystem exists for Assessment (pairing + upload, `Assets/_Project/Scripts/Assessment/`, 15 files) and for Evidence Inventory/Custody/HeldItem (`EvidenceCustodyLocation.cs`, `EvidenceInventory.cs`, `HeldItemCoordinator.cs`, `EvidenceInventoryUI.cs`, `EvidenceInventoryEntryView.cs`, `HeldEvidenceStorePrompt.cs`, `EvidenceBoxDepositZone.cs`), plus three new Testing files and a disabled `DevDiagnosticOverlay.cs`.

Why this matters more than usual: none of the above is committed. If the working tree is ever discarded (`git checkout .`, `git reset --hard`, a bad rebase, a stash that gets lost) all of it is gone. This is the single biggest immediate risk to the project as it stands. See Section 18 for a recommended commit-grouping strategy — do not execute it without the project owner's sign-off, this manual only recommends.

Completed / working systems (per source-level verification this pass): evidence lifecycle state machine and its tool integrations (swab, fingerprint kit/tape, photograph, recorder, evidence bag, chalk, tents); requirement tree evaluator (AND/OR/NOT); relationship/objective/narrative-event/hypothesis-checkpoint/ending systems and their causal chain; session event logging to local JSONL; assessment DTO construction and upload queue with retry/backoff; pairing-code exchange against the real backend (confirmed end-to-end this session, including a real network round trip); backend API routes, RLS policies, and scoring pipeline entry point (`verifySession`).

Partially complete / dormant: the deduction corkboard (`EvidenceBoardController`/`DeductionScorer`) is fully implemented but **not placed in any scene or prefab** (confirmed by the deduction-system audit and by that code's own comment) — meaning `RelationshipStateManager.CompleteAllSelected()` (needed for `RelationshipCompleted` requirement leaves, which the top-priority "wife convicted" ending depends on) currently has no reachable call site in the live game. Evidence tent pickup/reclaim (`EvidenceTentPickup`/`EvidenceTentPickupCoordinator`) is fully implemented but unreachable because `EvidenceTentTool.TentPickupEnabled` is hardcoded `false`. `AssessmentUploadConfig_QuestDev.asset` and `AssessmentUploadConfig_Production.asset` both have non-functional `apiBaseUrl` values (an unreplaced placeholder and an empty string, respectively) and will fail closed on any real device build pointed at either.

Known blockers/risks (see Section 21 for the full risk matrix): the `PreSessionFlowController` development-bypass gate activates on ANY `Debug.isDebugBuild` (not just the Editor), which includes ordinary Quest "Development Build" sideloads used for QA — a real risk of accidentally shipping a QA build that skips pairing entirely. `Assets/Resources/DevAgentSettings.asset` has a literal `accessToken` value committed to the repo (flagged as a hygiene concern, not confirmed as a live secret). The project's Android bundle identifier is still the unmodified URP template default (`com.UnityTechnologies.com.unity.template.urpblank`) — this must change before any real store submission.

Recommended next task (synthesized from the above, NOT an instruction the project owner has confirmed): commit the two new subsystems and this session's bug fixes as separate, reviewable commits (see Section 18), then either wire the deduction board into the scene or replace the "wife convicted" ending's `RelationshipCompleted` leaves with an evaluable alternative, since as authored that ending may be currently unreachable.

2. Repository Structure

2.1 Top-level layout

```
My project (1)/                      Unity project root
■■■ Assets/
■   ■■■ _Project/                     ← the actual game (everything else in Assets/ is
■   ■                                    template scaffolding, third-party art/asset-store
■   ■                                    packages, or Unity/Meta package samples)
■   ■■■ Resources/                    Global Resources folder — loose SO settings assets
■   ■                                    (Meta XR SDK config, DevAgentSettings, InputActions)
■   ■■■ Editor/                       Top-level (non-_Project) editor tooling
■   ■                                    (McpHouseBuilder, OpenableFurnitureSetup,
■   ■                                    ScenarioGroundTruthExporter, SceneLightingRealismSetup)
■   ■■■ XR/, XRI/, Oculus/            OpenXR / XR Interaction Toolkit / Meta Oculus
■   ■                                    integration settings and runtime support
■   ■■■ #Meta/                        Untracked — session/guid id files, likely written
■   ■                                    by AI-agent tooling (see 2.3)
■   ■■■ Plugins/Android/, StreamingAssets/, TextMesh Pro/, Samples/
■   ■■■ Scenes/, Settings/, Sky/, Screenshots/, Decor/, TutorialInfo/,
■   ■   Unity UI Samples/            Stock Unity template / sample content — not
■   ■                                    project-authored, safe to ignore
■   ■■■ (HDRP) NYC-Like City.../, Barking_Dog/, FurnishedCabin/,
■   ■   Furniture Mega Pack/, ..., ModularHousePack1/, ScifiOfficeLite/
■   ■                                 Third-party art/asset-store packages — never edit
■   ■■■ _Recovery/                    Unity crash-recovery scene backups
■■■ Packages/                         manifest.json / packages-lock.json (see 2.2)
■■■ ProjectSettings/                  Player/Build/Graphics/etc. settings (YAML)
■■■ Tools/Audit/                      Node-based read-only audit tooling (see 2.4)
■■■ docs/                             Prior documentation — READ SECTION 2.5 FIRST,
■                                      it is stale on two major subsystems
■■■ VR-CSI_SESSION_HANDOFF.md         Append-only session handoff log (also stale on
                                       the same two subsystems — see 2.5)
```

2.2 `Assets/_Project/` — the actual project

Folder	Purpose	Notes
`Art/`	3D models, textures, materials, icons	`Animation/`, `Environments/MODELS/`, `ICONS/ICONS/`, `Materials/`, `Meshes/`, `Props/Materials/`
`Audio/`	SFX and voice-line audio assets	`SFX/`, `VoiceLines/`
`Data/`	ScriptableObject-driven case content — the authoring layer	See Section 5 in full
`Editor/`	Project-specific editor tooling	Only `Requirements/RequirementDefinitionEditor.cs` + `RequirementNodeDrawer.cs` — no other `Editor/` folders exist anywhere under `Assets/_Project/Scripts/`
`Materials/`	Runtime materials	Mostly `UI/` subfolder; sparse otherwise
`Prefabs/`	Prefab library	`Evidence/`, `Interaction/`, `Tools/`, `UI/`
`Scenes/`	Playable scenes	See Section 2.6
`Scripts/`	All C# gameplay/system code, organized by domain	See Section 3 in full — 137 files, 18 subfolders
`UI/`	UI prefabs/assets	`HUD/`, `Menus/`

What should NOT belong in `_Project/`: third-party art, asset-store packages, or anything not authored for this game — those all live at the `Assets/` top level, outside `_Project/`, by existing convention. Keep it that way; it makes "what did we actually write" trivially answerable by `find Assets/_Project`.

2.3 Packages and project configuration

Unity version: 6000.5.6f1 (per task brief; not independently re-verified against `ProjectVersion.txt` this pass).

Key packages (`Packages/manifest.json`, VERIFIED):

Package	Version	Role
`com.meta.xr.sdk.all`	205.0.0	Meta XR SDK umbrella (pulls core/audio/voice/haptics/mrutilitykit/platform/interaction/interaction.ovr, all 205.0.0 except audio 85.0.0/voice 85.0.1)
`com.unity.xr.openxr`	1.17.1	OpenXR runtime
`com.unity.xr.interaction.toolkit`	3.5.1	XR Interaction Toolkit
`com.unity.xr.hands`	1.9.0	Hand tracking
`com.unity.xr.management`	4.7.0	XR provider management
`com.unity.inputsystem`	1.20.0	New Input System (all controller bindings in this project use raw loose `InputActions` against this)
`com.unity.render-pipelines.universal`	17.5.0	URP
`com.unity.textmeshpro`	3.0.6 (transitive, via Meta packages — not a direct manifest entry)	TMP
`com.unity.ai.inference`	2.6.1	Unity Sentis (on-device inference; `SENTIS_ANALYTICS_ENABLED` scripting define is set)
`com.unity.ai.assistant`	2.17.0-pre.1	Unity's in-Editor AI Assistant (pre-release)
`com.coplaydev.unity-mcp`	git, `main` branch, `/MCPForUnity` subpath	**MCP for Unity** — exposes the Editor to Model Context Protocol clients; this is the mechanism this very manual was produced through

Android build config (`ProjectSettings/ProjectSettings.asset`, VERIFIED): IL2CPP scripting backend, arm64-v8a only, minSdk 32 (Android 12) / targetSdk 34 (Android 14), .NET Standard 2.1 API compatibility, no release keystore configured (debug signing only), code minification off. `productName: My project (1)`, `companyName: DefaultCompany`, Android `applicationIdentifier: com.UnityTechnologies.com.unity.template.urpblank` — **the project has never been rebranded off the URP Blank template defaults; this must change before any real store submission.**

XR/OpenXR config (`Assets/XR/Settings/OpenXRPackageSettings.asset`, VERIFIED): Android render mode is **Single Pass Instanced**. Feature toggles enabled for Android: Meta XR Feature bundle, foveated rendering/subsampled layout, and several controller/hand-interaction profiles (KHR Simple, Microsoft Hand Interaction, Hand Interaction, Android Mouse, Oculus Touch Proximity, Quest Touch Pro/Plus). Notably, the **Hand Tracking subsystem feature itself is disabled** for Android even though hand-interaction controller profiles are enabled — worth confirming with the project owner whether that's intentional. An OpenXR API layer named `XR_APILAYER_METAX_operator` ("API layer that providing agentic interface for OpenXR") is registered and enabled for the Standalone platform group — consistent with the project having AI-agent tooling wired in beyond just this manual-writing session (see next paragraph). Its DLL was not located as an actual file in this pass — NOT VERIFIED what installs it.

AI/agent tooling embedded in the project (worth knowing, not just this session's own access): `com.coplaydev.unity-mcp` (above), `Assets/Resources/DevAgentSettings.asset` (Meta Immersive Debugger "Dev Agent" — configures its own `mcpServerPort`/`serverPort`/`serverAddress`; **currently** `enabled: 0`), and the untracked `Assets/#Meta/session_id.id/guid.id` files. **Hygiene flag**: `DevAgentSettings.asset` has a literal `accessToken` value committed in the asset. This may be a harmless local/dev-only token, but it should be reviewed and likely rotated/removed before any public repository sharing — do not assume it is safe.

Editor tooling: `Assets/Editor/` (top-level) holds four scripts inferred from name only (not opened this pass): `McpHouseBuilder.cs`, `OpenableFurnitureSetup.cs`, `ScenarioGroundTruthExporter.cs`, `SceneLightingRealismSetup.cs`. `Assets/_Project/Editor/Requirements/` holds the two requirement-system editor scripts named above.

2.4 `Tools/Audit/` — Node-based read-only audit tooling

Confirmed present and runnable (plain Node ESM, no `package.json`/npm deps needed — only `node:fs`/`node:path`/`node:crypto`/`node:child_process`). Not wired to any CI found in this pass.

Script	Role
`inspect.mjs`	Read-only repo inventory: hashes first-party source/asset files, cross-checks `.meta` GUIDs against references for unresolved-reference detection, scans for TODO/FIXME/mock markers, greps for credential-like patterns. Outputs `docs/evidence/repository-inspection.json`.
`inspect-content.mjs`	Targeted content-consistency scan (found the Main-scene broken UnityEvent callbacks referenced in prior docs). Outputs `docs/evidence/content-consistency.json`.
`create-traceability.mjs`	Generates the requirements traceability matrix from an external requirements-spec text file (kept deliberately outside this repository, per its own comment).
`create-test-plan.mjs`	Generates the acceptance-test-plan rows.
`finalize-reports.mjs`	Cross-checks Unity test-suite output logs (`docs/evidence/*.txt`) against expected counts to assemble the final report numbers.
`scorer.test.mjs`	Node test-runner fixtures for `Tools/EvidenceScorer` (a separate .NET scoring CLI).

`docs/evidence/` holds 13 raw output artifacts from these tools (repository-inspection.json, requirements.json, android-build-result.json, unity-scene-health.txt, etc.), all dated 2026-09-14/15 — i.e. from the same stale baseline as the docs below, predating the Assessment and Inventory subsystems.

2.5 Prior documentation — read this before trusting anything else in `docs/`

Six loose Markdown files exist in the repo (confirmed exhaustive by repo-wide search, excluding `Library/`, `node_modules/`, and Unity package samples): `VR-CSI_SESSION_HANDOFF.md` (root), `docs/VICSI_ENGINEERING_REPORT.md`, `docs/VICSI_IMPLEMENTATION_AUDIT.md`, `docs/VICSI_REQUIREMENTS_TRACEABILITY_MATRIX.md`, `docs/VICSI_TEST_REPORT.md`, `docs/superpowers/specs/2026-08-30-camera-aim-feedback-design.md`.

All five status-report docs (everything except the camera-aim design spec) were produced at or just after commit `0ab53f2` and say nothing about the Assessment/pairing/upload system or the Evidence Inventory/Custody/HeldItem system — neither existed yet. Specifically: `VR-CSI_SESSION_HANDOFF.md` (1013 lines, read in full this pass) never mentions "Assessment," "Inventory," "Custody," or "HeldItem" anywhere; the traceability

matrix's D-series (session logging/scoring) and E-series (Node API, all marked Missing) rows cite only `SessionLogger.cs`, never the new `Assessment/` files that plausibly now implement several of those E-series requirements. **Do not use these five docs as a source of truth for pairing, upload, inventory, or custody behavior — use this manual and the live source instead.** They remain useful, accurate history for the evidence-procedure lifecycle, chalk tracing, and the RecorderReportUI redesign (their strongest, still-current content).

3. GameObject / Scene Architecture

Captured directly from the live Editor's scene graph for `CSI_Environment.unity` on 2026-09-19 (VERIFIED — not parsed from raw YAML). 46 root GameObjects total.

3.1 `_Managers` — the single most important GameObject in the project

`_Managers` holds **25 sibling** `MonoBehaviour` **components** on one GameObject, not 25 separate GameObjects:

```
EvidenceStateManager, RoleSceneLoader, ProceduralGateValidator, STCSManager,
NotificationManager, EvidenceNotifier, PhotoAlbumManager, LocalJsonLogWriter,
SessionLogger, HypothesisCheckpointUI, HypothesisCheckpointManager,
InteractionFeedback, FeedbackDirector, GreyboxFlowTest, NotificationPriorityTest,
EndingManager, RelationshipStateManager, NarrativeEventManager,
ObjectiveStateManager, AssessmentUploadService, PairingIdentityService,
PreSessionFlowController, EvidenceInventory, HeldItemCoordinator,
HeldEvidenceStorePrompt
```

Why this exists: virtually every singleton/manager-style system in the game (Section 3.4 in the Script Catalog explains each one) is deliberately a scene-manager `MonoBehaviour` with a static `Instance` property, not a `DontDestroyOnLoad` cross-scene singleton or a pure static class — this project's own convention (per `HeldItemCoordinator`'s doc comment) is that new manager-style code should be a scene-manager component, with `PlayerToolRegistry`/`PlayerUIGate` (pure static classes) being legacy exceptions that predate the convention.

What breaks if `_Managers` **is removed, disabled, or duplicated**: everything. Evidence tracking, session logging, requirement evaluation, the entire narrative/objective/ending chain, notifications, the assessment upload pipeline, and pairing all stop functioning simultaneously. **This object must never be deleted, deactivated, or duplicated.** If you need a second instance for testing, every Testing-folder script that does this explicitly saves and restores the real `Instance` via reflection in a `finally` block (see Section 17) — do the same.

`_Managers` has 3 child GameObjects (not components): `FingerprintStation`, `CompletionScreen`, `PairingUICanvas`.

3.2 Other root-level GameObjects (selected — full list of 46 roots captured; only gameplay-relevant ones detailed here)

GameObject	Components (beyond Transform)	Role
`[BuildingBlock] Camera Rig`	`BuildingBlock`, `OVRCameraRig`, `OVRManager`, `OVRHeadsetEmulator`, `PlayerHeightOffset`	The player — a Meta XR SDK "Building Block" rig. There is no separate "Player" hierarchy; this IS the player.
`ToolWheelController`	`ToolWheelController`	Radial tool-select menu (see Section 11)
`UtilityMenu`	`UtilityMenuController`	Radial utility menu (Notes/Photo Album/Evidence Inventory/...)
`Pointable Canvas Module`	`EventSystem`, `PointableCanvasModule`, `BaseInput`	Meta ISDK ray→uGUI routing — present, but this project's own UI (Section 11) deliberately bypasses it via raw `InputAction` polling instead, after confirming it never worked reliably against runtime-built World Space canvases
`PhotographerCamera`	`EvidenceProp`, `PhotographTool`, `Light`, `AudioSource`, `PhotoCaptureListener`	The camera prop. Notably ALSO registered as an `EvidenceProp` itself (grabbable/interactable), an oddity worth knowing about if you ever iterate `EvidenceProp.All`.
`IOC_UVLight`	`EvidenceProp`, `FlashlightTool`, `Light`	UV flashlight — also an `EvidenceProp`

GameObject	Components (beyond Transform)	Role
`EvidenceBag`, `SwabTool`, `FingerprintKit`, `FingerprintTape`, `ChalkMarkTool`, `EvidenceTentDispenser`, `LetterTentDispenser`	`PlayerTool` subclasses each	Equippable tools (Section 6/9)
`Evidence_KitchenKnife`, `Evidence_Bloodspatterpattern`, `Evidence_Victimsmobilephone`, `Evidence_Emptyliquorbottle`, `Evidence_DisposableGloves`, `BODY_100K`	`EvidenceProp` (+ `EvidenceGrabGate`/`RightHandOnlyFilter` on the pickupable ones)	The scenario's evidence roster (EVD-014, EVD-015, EVD-017, EVD-018, EVD-019, BODY-A)
`EvidenceHoldingCrate`	`EvidenceBoxDepositZone`	Where sealed evidence gets deposited from inventory (new system)
`EvidenceInventoryPanel`	`Canvas`, `GraphicRaycaster`, `EvidenceInventoryUI`, `HudFollowCamera`, `PointableCanvas`	**Inactive by default** (`activeSelf: false`) — the sealed-evidence inventory panel; `PointableCanvas` was added this session as part of a (since-superseded, see Section 22 changelog) VR-ray debugging pass
`NotificationCanvas`, `STCSNotificationCanvas`	`Canvas`, `CanvasGroup`, `HudFollowCamera`, `NotificationUI`/`STCSNotificationUI`	Two separate toast systems — general notifications vs. STCS narrative beats. Both use `HudFollowCamera`; this session fixed a bug where both had identical `LocalOffset` values and would overlap at runtime despite looking separated in the static Editor view.
`RecorderDictaphone`, `RecorderClipboard`, `RecorderReport`	`RecorderTool`; plain Transform; `RecorderReportUI`	Recorder tool's pen (main object), clipboard (detached standalone root, this session's redesign), and its report-writing panel controller
`ChalkGuideTemplate`	`EvidenceChalkGuide`	**Inactive** — 48 children — the template chalk-outline geometry, cloned per evidence item at runtime
`Harong`	(87 children, `isStatic: true`)	Large static environment art (house/room shell)
`Road`, `Fence_2`/`Fence_5` (×7)	Static mesh/collider only	Environment art, no gameplay scripts

3.3 Common component signature for pickupable evidence

Every grabbable evidence prop shares this exact component set:

```
BoxCollider, Rigidbody, Grabbable, HandGrabInteractable, GrabInteractable,
EvidenceProp, ToggleGrab [, EvidenceGrabGate, RightHandOnlyFilter]
```

The last two (`EvidenceGrabGate`, `RightHandOnlyFilter`) only appear on the "personal effects" tier that needs physical pickup gating (knife, phone, bottle, gloves) — not on static/non-pickup evidence like the blood spatter or the body, which use `EvidenceProp` alone.

3.4 Cross-reference: what to do if you need to see the FULL live hierarchy yourself

Use Unity MCP's `manage_scene` tool (`action: get_hierarchy`) or, without MCP, open the scene in the Editor and expand the hierarchy manually. A raw YAML read of `CSI_Environment.unity` is possible but the file is large (9,089 uncommitted diff lines alone) and GameObjects are addressed by opaque `fileID`s in that format — the live Editor/MCP

route is far more practical for exploration.

4. Script Catalog — CaseFile, RoleSystem, ProceduralGate, Core

`Assets/_Project/Scripts/Core/` **does not exist in this checkout** (confirmed via directory search) — if you were told to look there, that information is stale; there is nothing to document.

4.1 CaseFile folder (11 files) — evidence identity, state, and custody

`EvidenceStatus.cs` — enum, THE procedural lifecycle

```
NotFound=0, Found=1, PhotographedRaw=2, ChalkMarked=3, Marked=4,
PhotographedFinal=5, Logged=6, ReadyForCollection=7, Collected=8,
Sealed=9, Processed=10
```

Order authority is `EvidenceStateManager.RequiredSequence` (a separate private array), not this enum's own declaration order — kept deliberately independent even though they currently match, specifically so a future reorder of the enum can never silently re-derive a wrong sequence. **ORDER-SENSITIVE — append only, never reorder or renumber.** The source file carries its own "insertion log" documenting at least three past incidents where inserting a member shifted later ordinals and required hand-correcting `.asset` YAML.

The real procedure: examiner photographs the undisturbed scene (`PhotographedRaw`) → chalks an outline (`ChalkMarked`) → places a numbered tent (`Marked`) → second photo with the tent in frame (`PhotographedFinal`) → documents in the recorder (`Logged`) → auto-advances to `ReadyForCollection` (unless `examinedInPlace`) → bagged (`Collected`) → sealed (`Sealed`) → auto-advances to `Processed` once any fingerprint requirement clears.

`EvidenceStateManager.cs` — THE core class (1146 lines)

`MonoBehaviour`, singleton (`Instance`). Lives on `_Managers`. Also declares `TransitionResult{Applied,Duplicate,Violation}`, `ReclaimResult{Reverted,BlockedDocumented,NoChange}`, `FingerprintLabStatus{NotSent=0,SentToLab=1,Matched=2}` (a fully **decoupled side-track** — never interacts with the main status chain; only `MarkSentToLab`/`MarkMatched` touch it).

Every `Mark*` transition method (each called by the tool that performed the action, each returns `TransitionResult`): `MarkFound`, `MarkTented`(numbered), `MarkTentedWithLetter`(for `examinedInPlace` items), `MarkPhotographedRaw`, `MarkChalkMarked`, `MarkPhotographedFinal`, `MarkLogged` (auto-chains to `CheckReadyForCollection`), `MarkCollected`, `MarkSealed` (auto-chains to `CheckProcessed`), `MarkProcessed`, `MarkFingerprinted` (also auto-calls `MarkSentToLab`), `MarkSentToLab`, `MarkMatched` (stub — sets a hardcoded placeholder match target), `MarkSwabbed`, `MarkFlipped`, `TryReclaimMarker` (reverts a `Marked` item back one step; **the original log event is never deleted**, a `MarkerReclaimed` event is appended beside it — scoring must read full event history, not a final-state snapshot).

Every transition funnels through one private gate, `SetStatus`, which: rejects unknown evidence ids; rejects statuses outside `RequiredSequence`; blocks `examinedInPlace` items past `Logged`; treats a same-or-backward call as harmless `Duplicate` (not logged as a violation) and a skipped-ahead call as a `Violation` (fires `OnEvidenceTransitionBlocked`); re-checks the same three per-item blocking rules `CanAdvanceTo` already checked (defense-in-depth against a tool calling a `Mark*` method directly without pre-checking); and on success updates the record, fires `OnEvidenceStatusChanged`.

Per-item blocking rules (`GetItemBlockReason`, checked in order): `IsBlockedByInPlaceExamination` → `IsBlockedByFingerprinting` (gates `Processed` for the default post-collection fingerprint shape) → `IsBlockedByOnSceneFingerprinting` (gates `Logged` instead, for items like the gloves that need on-scene fingerprinting) → `IsBlockedBySwabbing` (gates `Logged`).

Public queries: `GetRecord`, `IsAtOrPastStatus` (index-comparison via `RequiredSequence`, never a raw enum-int compare), `CanAdvanceTo`, `GetItemBlockReason`, `CountAtOrAbove` (excludes `examinedInPlace` items),

`IsFingerprintMatched`, `IsFingerprintLabStatusAtOrAbove`, `HasSelectedReportLine`, `RecordReportLinesSelected`.

Static events: `OnEvidenceStatusChanged`, `OnEvidenceTransitionBlocked`, `OnEvidenceSwabbed`, `OnEvidenceFingerprinted`, `OnFingerprintLabStatusChanged`, `OnReportLineSelected`. These six events are the backbone the rest of the game hangs off — 35 files across the codebase reference `EvidenceStateManager.Instance` (confirmed via whole-tree grep); the confirmed direct consumers include `EvidenceInventory`, `EvidenceBoxDepositZone`, `EvidenceBagTool`, `ChalkMarkTool`, `EvidenceProp`, `SelectionPrerequisite`, `RequirementEvaluator`, `HypothesisCheckpointManager`, `SessionLogger`, `AssessmentSessionBuilder`, and (by name-cross-reference, not independently opened this pass) `FingerprintKitTool`, `SwabTool`, `RecorderTool`, `PhotographTool`, `EvidenceTentTool`, `LetterTentTool`, `EvidenceFlipTool`, `EvidenceGrabGate`, `UVRevealable`, `STCSManager`.

`EvidenceRecord.cs` — plain `[Serializable]` class, NOT a ScriptableObject

The runtime mutable state for one evidence item, held in `EvidenceStateManager`'s internal dictionary (keyed by `evidenceId`; the record itself carries no id field). Fields: `definition` (link to `EvidenceDefinition`), `status`, `lastToolUsed`, `statusChangedAtTime`, `custodyLocation` (see 4.3), `fingerprintingDone`, `swabbingDone`, `isFlipped`, `tentNumber`/`tentLetter`, `selectedReportLineIds` (HashSet, never cleared once set), `fingerprintLabStatus`, `fingerprintLabMatchResult`.

`EvidenceDefinition.cs` — ScriptableObject, the authoring asset

`[CreateAssetMenu(menuName = "VR-CSI/Evidence Definition")]`. The per-item content asset a designer edits (e.g. `EVD-014_KitchenKnife.asset`). Full field list is in Section 5 (Data Model) — it configures identity, interaction radius, fingerprint/swab/flip preconditions, chalk-guide sizing, report-line candidates (nested `ReportLineOption`), and the held-in-hand visual (`heldPrefab` + local transform).

`EvidenceCustodyLocation.cs` (new/uncommitted) — enum, a SEPARATE axis from `EvidenceStatus`

`Scene=0, Inventory=1, Held=2, EvidenceBox=3`

Tracks **where** an item physically is right now, completely independent of **what procedural step** it has reached (`EvidenceStatus`). Only ever assigned by `EvidenceInventory`. This distinction — procedure-step vs. physical-location — is one of the most important conceptual splits in the whole codebase; see Section 8.

`EvidenceInventory.cs` (new/uncommitted) — MonoBehaviour singleton on `_Managers`

Owns evidence custody and "the evidence half of what's in the player's hand." Deliberately does not decide mutual exclusion with tools — that is `HeldItemCoordinator`'s job (see 4.2). `Add` (Scene→Inventory, requires `Sealed`+), `EquipHeld` (Inventory→Held, requires a `heldPrefab`, refuses otherwise), `UnequipHeld` (Held→Inventory), `Deposit` (Held→EvidenceBox, terminal). Fires `OnInventoryChanged`/`OnCustodyChanged`, logs `SessionEventType.EvidenceAddedToInventory`/`EvidenceEquipped`/`EvidenceUnequipped`/`EvidenceDeposited`. Full detail in Section 8.

`EvidenceProp.cs` — MonoBehaviour on every physical evidence GameObject

Marks a scene object as an identifiable evidence item; owns automatic `Found` detection via a runtime-created trigger `SphereCollider` sized to `EvidenceDefinition.interactionRadius` (unless `foundRequiresExternalTrigger`, e.g. a UV-only print). Static registry (`EvidenceProp.All`), not an `Instance` singleton. Also carries `SurfaceNormalAxis` for chalk-guide orientation (since `transform.up` isn't reliably "outward" across the art asset roster).

`EvidenceFingerprintMarker.cs` / `EvidenceSwabRequirement.cs` — precondition markers, not separate collectible items

Both mark a physical location on a PARENT evidence item (a latent print; a blood trace) that must be dusted+lifted, or swabbed, before the parent can proceed — neither is its own `EvidenceRecord`. `EvidenceSwabRequirement`'s own doc comment explains this replaced an earlier broken design where the blood trace was its own co-located `EvidenceProp`, which the camera's frame-based targeting couldn't reliably disambiguate from its parent (see Section 6, `PhotographTool`).

`EvidenceRelevance.cs` — enum, scoring metadata only

`Critical=0, Relevant=1, Neutral=2, Distractor=3` — **ORDER-SENSITIVE, append-only.** Deliberately never read at interaction time (would leak the correct-answer identity to the player); read only post-session by scoring.

`SelectionPrerequisite.cs` — the pre-`RequirementNode` cross-item unlock mechanism, still actively used

`PrerequisiteType{ReportLineSelected(default), FingerprintMatched}`. `SelectionPrerequisite.EvaluateOne` is "the single authoritative place one entry is evaluated" (its own doc comment says do not reimplement this switch elsewhere) — also reused verbatim by `RequirementEvaluator`'s `LegacySelectionPrerequisite` leaf type. Not deprecated; new authoring that needs AND/OR/NOT nesting uses `RequirementNode` instead (Section 9), which can wrap a legacy list via `RequirementNode.FromLegacyList`.

4.2 RoleSystem folder (9 files) — tool equip machinery

`PlayerTool.cs` — abstract base for every equippable tool

`[RequireComponent(typeof(Grabbable))]`. Every tool is virtually-equipped from the tool wheel (hidden/inert renderer+colliders until equipped — see `SetEquippedVisualState`), not a physical world pickup. `abstract ToolType ToolRole`; `virtual Hand PreferredHand => Right` (only `EvidenceBagTool` overrides to `Left`); `EquipToHand`/`Holster` (virtual, many tools override to also move a second off-hand prop — see the two-handed pattern in Section 6); `ReportEvidence(evidenceId, markMethod, stcsSuffix)` helper that calls an `EvidenceStateManager` mark method and optionally fires an STCS trigger.

`ToolType.cs` — enum, explicit values

```
None=0, Photographer=1, IOC=2, Sketcher=3, EvidenceCollector=4, Recorder=5,
EvidenceMarker=6, Swab=7, FingerprintKit=8, ChalkMark=9, EvidenceFlip=10,
FingerprintTape=11, LetterTentMarker=12
```

ORDER-SENSITIVE, append-only — explicitly cited by `EvidenceStatus.cs`'s own comment as a prior real incident of the same bug class. `Sketcher` (3) has no corresponding tool script found in `Interaction/` — NOT VERIFIED whether it exists elsewhere. `TeamLeader`/`CaseAnalyst` existed historically and were removed (neither is tool-mediated in the unified single-investigator design).

`PlayerToolRegistry.cs` — static class, directory of every tool in the scene

NOT an `Instance` singleton — pure static, predates the scene-manager convention. `CurrentRole`/`CurrentTool` (physically/virtually in-hand), `VirtuallyEquippedRole`, `AllTools`, events `OnToolGrabbed`/`OnToolHolstered`/`OnToolRegistered`. `ToggleEquip(role, handAnchor)` is the wheel's real equip path. `[RuntimeInitializeOnLoadMethod(SubsystemRegistration)]` clears all static state at the start of every Play session (guards against Unity keeping statics alive across scene loads).

`HeldItemCoordinator.cs` (new/uncommitted) — THE single authority for "what's in the player's hand"

MonoBehaviour singleton on `_Managers`. Unifies tools AND evidence: no controller (`ToolWheelController`, `EvidenceInventoryUI`) independently decides what's held — both must go through this. Owns no cached state;

`CurrentCategory`/`CurrentToolRole`/`CurrentEvidenceId` are computed live by reading `PlayerToolRegistry` and `EvidenceInventory`. `ToggleEquipTool` deterministically unequips held evidence first; `EquipEvidence` holsters the current tool first. This is the **current correct integration point** — treat it as superseding direct calls to `PlayerToolRegistry`/`EvidenceInventory` for equip/unequip logic anywhere new code is added.

`ToolWheelController.cs` (880 lines) — the radial tool-select menu

Hold left-controller Y, tilt right thumbstick, release to equip. Rebuilds its wedges fresh from `PlayerToolRegistry.AllTools` every open (a new `PlayerTool` shows up automatically; unbuilt roles show dimmed placeholders). Fully covered in Section 11.

`LocomotionSuspender.cs` — static utility

Reference-counted `Suspend()`/`Resume()` shared by every exclusive UI panel to pause walking/turning while it's open.

`PlayerHeightOffset.cs` — shifts the `OVRCameraRig`'s tracking-space height (Floor Level tracking convention).

4.3 ProceduralGate folder (1 file)

`ProceduralGateValidator.cs` — generic transition gate, thin wrapper over `EvidenceStateManager`

MonoBehaviour singleton on `_Managers`. `CanTransition`/`GetBlockReason` delegate to `EvidenceStateManager.CanAdvanceTo`/`GetItemBlockReason`. Maintains an in-memory `EventLog` of every transition seen (comment notes a future `evidence_events` Supabase table mirror — not built). Subscribes to `OnEvidenceStatusChanged`.

5. Script Catalog — Requirements (the shared evaluator)

`Assets/_Project/Scripts/Requirements/` (5 files) — the generalized AND/OR/NOT evaluator that everything downstream (relationships, objectives, narrative events, submission gating, ending resolution, hypothesis-checkpoint eligibility) is built on top of.

`RequirementEnums.cs`

```
RequirementNodeKind:        Leaf=0, Group=1
RequirementLogicalOperator: All=0, Any=1, Not=2   (Not is unary: exactly one child)
RequirementType:            EvidenceStatusAtOrAbove=0, ReportLineSelected=1,
                             FingerprintLabStatusAtOrAbove=2, EvidenceSwabbed=3,
                             EvidenceFingerprinted=4, EvidenceFlipped=5,
                             EvidenceDiscovered=6, LegacySelectionPrerequisite=7,
                             RequirementAssetReference=8, NarrativeEventCompleted=9,
                             ObjectiveCompleted=10, RelationshipCompleted=11
```

All three **ORDER-SENSITIVE, append-only** — persisted as raw ints in every `RequirementNode` embedded in every `.asset` in `Data/`. `RequirementType`'s doc comment states this explicitly: "the same discipline EvidenceStatus/ToolType/SessionEventType already follow." Each value's authoritative source: `EvidenceStatusAtOrAbove`→`EvidenceStateManager.IsAtOrPastStatus`; `EvidenceDiscovered`→shorthand for `IsAtOrPastStatus(Found)`; `NarrativeEventCompleted`→`NarrativeEventManager.IsCompleted`; `ObjectiveCompleted`→`ObjectiveStateManager.IsCompleted` (this is also how "depends on an earlier objective" is expressed — no separate dependency-list field exists); `RelationshipCompleted`→`RelationshipStateManager` record state.

`RequirementNode.cs` — one node in a requirement tree

`[System.Serializable] class RequirementNode`, recursive via `[SerializeReference] List<RequirementNode> children` — `SerializeReference` is used specifically because Unity's default serializer statically unrolls nested `[Serializable]` value types to a hard depth cap of 10, confirmed hit live ("Serialization depth limit 10 exceeded"), which would silently drop deeper trees with only a console warning, never a build error.

Default value is load-bearing and was a real shipped bug: `kind` defaults to `Group`/`All` with empty children (vacuously TRUE), never `Leaf`. A bare `new RequirementNode()` must mean "no requirement configured" = pass. Defaulting to `Leaf` was a real, live bug once: a fresh Leaf defaults to `requirementType=EvidenceStatusAtOrAbove` with a null `evidenceId`, which fails closed — every unconfigured "extra requirement" field silently, always failed, confirmed live as hard-blocking `GreyboxFlowTest`'s report-line selection and silently suppressing hypothesis-checkpoint eligibility.

`static RequirementNode FromLegacyList(List<SelectionPrerequisite> legacy)` wraps a flat legacy list as an ALL-group of `LegacySelectionPrerequisite` leaves.

`RequirementEvaluator.cs` — static class, stateless, read-only (never calls a `Mark*` method)

Truth table (locked-in project policy): empty `ALL` → **true**; empty `ANY` → **false**; `NOT` with `children.Count != 1` → **fails closed** (checked before the generic empty-children branch — a prior buggy version let empty-NOT read as vacuously true, caught by a regression test before shipping).

`IsSatisfied(node)` / `Evaluate(node, includeChildResults=false)`. `EvaluateGroup` short-circuits `All` on first failure and `Any` on first pass (unless `includeChildResults`, which evaluates every child to populate a debug tree). `EvaluateLeaf` dispatches to 11 handler methods, one per `RequirementType`. `EvaluateAssetReference` recurses into a `RequirementDefinition` asset's root with a `HashSet<RequirementDefinition> visitedAssets` cycle guard — an ancestor-cycle fails with `CircularReference`, but a diamond-shaped shared reference (two branches legitimately

pointing at the same asset) still resolves normally, because the guard set removes an asset after its own subtree finishes.

`RequirementEvaluationResult.cs` / `RequirementDefinition.cs`

`RequirementEvaluationResult`: `Passed`, `Reason` (`RequirementFailureReason` enum — dev/debug-facing only; explicitly documented boundary: may reveal that a step hasn't happened, must never leak WHICH relationship/theory/ending is correct). `RequirementDefinition` (ScriptableObject) wraps one reusable named tree, referenced via `RequirementType.RequirementAssetReference` — `Assets/_Project/Data/Requirements/` currently has **zero authored** `.asset` **files**, so there is no concrete worked example in project data; every current tree is embedded inline in its owning field.

6. Script Catalog — Interaction (all tools; 34 files)

6.1 The generic two-handed equip pattern

Used by `SwabTool`, `FingerprintKitTool`, `RecorderTool` (clipboard). Shape:

1. A serialized off-hand `Transform` (e.g. `containerVisual`/`jarVisual`/`clipboardVisual`) plus the tool's own tip/receiving `Collider`.
2. `Awake()` caches the off-hand prop's original parent/pose and hides it (`PlayerTool.SetEquippedVisualState` only manages the MAIN GameObject's own children, not this separate object).
3. `override EquipToHand`: calls `base.EquipToHand` (right-hand prop as normal), **explicitly re-enables its own tip collider** (base class disables ALL colliders in both equip states), then reparents the off-hand prop onto `OVRCameraRig.leftHandAnchor`.
4. `override Holster`: reverses it.
5. A thin `*TipReceiver` forwarder component sits on the tip's small child trigger-collider, forwarding `OnTriggerEnter/Exit` up to the owning tool's public handler (trigger callbacks only fire on the GameObject the collider is actually on).

`FingerprintTapeTool` is the deliberate single-hand contrast case (its own comment calls this out: "no second prop to hold opposite the tape").

6.2 Catalog (all 34 files)

Script	Type	Role	Must-wire fields (no fallback)
`SwabTool.cs`	`PlayerTool` (Swab)	Click-based blood/trace sampling: touch tip + trigger to swab, carry to container + trigger to contain, again to seal. Distinguishes a real `EvidenceProp` sample from an `EvidenceSwabRequirement` marker via separate tracked fields.	`tipCollider`, `tipRenderer`, `containerVisual`, `containerReceivingZone`, `rightActivateAction`
`FingerprintKitTool.cs`	`PlayerTool` (FingerprintKit)	Two-handed dust brush + jar; click-to-load, contact-to-dust (no press needed), generalized over `EvidenceFingerprintMarker`. `TryLift` lives here even though the tape tool touches the print (all needed gate logic already lives here).	`brushTipCollider`, `brushRenderer`, `jarVisual`, `jarReceivingZone`, `rightActivateAction` — **both** `brushTipCollider` **AND** `BrushTipReceiver.fingerprintKitTool` **were found unwired this session, breaking the entire dust-load/dust-apply gesture; see Section 22 changelog**
`FingerprintTapeTool.cs`	`PlayerTool` (FingerprintTape)	Separately-equippable lifting tape; single right-hand item; forwards contact straight to `FingerprintKitTool.Instance.TryLift`.	`tipCollider`

Script	Type	Role	Must-wire fields (no fallback)
`FingerprintStationTool.cs`	plain `MonoBehaviour`	Fixed-station proximity interaction for the default post-`Sealed` fingerprint shape. **Largely dead code as of this build** — both items that used to route through it (EVD-014, EVD-019) now go through FingerprintKit+Tape instead; kept for any future item re-added to `restrictToEvidenceIds`.	—
`PhotographTool.cs`	`PlayerTool` (Photographer)	Grabbable camera, raise-to-eye aiming (left X), frame-based (not pure-raycast) targeting. One shutter action produces `PhotographedRaw` or `PhotographedFinal` depending on the item's current status. Comment block headed "AMBIGUOUS CALLS MADE HERE" documents 3 deliberate-but-debatable targeting rules (frustum-on-pivot-only, occlusion-within-own-hierarchy, closest-to-viewport-CENTER not closest-in-3D — the last one a confirmed fixed bug).	`viewfinderOverlay`, at least one of `leftActivateAction`/`rightActivateAction`
`RecorderTool.cs`	`PlayerTool` (Recorder)	Aim-and-activate (raycast) recorder; opens `RecorderReportUI` for the aimed item (gated on `PhotographedFinal`); confirming reports `Logged`. Falls back to an instant log for any item with no authored report lines. Documented historical bug: a stripped-shader `Awake()` throw silently left the tool permanently unregistered in device builds only — now guarded.	`clipboardVisual`, activate actions
`EvidenceBagTool.cs`	`PlayerTool` (EvidenceCollector)	**Only tool overriding `PreferredHand` (Left)**. Physically embodied two-handed gesture: right-hand GRIP closes on evidence while left holds the bag open (TRIGGER); releasing left trigger seals. Reads its own grip action rather than Oculus's `SelectingPointsCount`, deliberately, for testability.	`receivingZoneCollider`
`EvidenceGrabGate.cs`	plain `MonoBehaviour`, `[RequireComponent(EvidenceProp)]`	Keeps an item's `HandGrabInteractable`/`GrabInteractable` disabled until genuinely `ReadyForCollection` AND swab/fingerprint preconditions clear — enforces "can't grab" at the physical layer too, not just the state machine.	—

Script	Type	Role	Must-wire fields (no fallback)
`EvidenceBoxDepositZone.cs` (new)	plain `MonoBehaviour`	Deposit exactly one held+eligible item per right-A press while inside its trigger (entering alone deposits nothing). Re-validates eligibility on every prompt refresh AND every press, never cached.	`triggerCollider` (warned via `OnValidate`)
`EvidenceFlipTool.cs`	`PlayerTool` (EvidenceFlip)	Minimal aim-and-activate flip (currently only meaningful for the gloves). Comment documents a previously-missing feedback gap (no toast on success) now fixed.	activate actions
`ChalkMarkTool.cs`	`PlayerTool` (ChalkMark)	**The project's only continuous/real-time interaction** — while trigger held, checks the chalk tip's position every frame against the nearest active `EvidenceChalkGuide`. Distance-based, not physics-collision, deliberately (avoids VR tunneling against a thin ring of colliders).	`chalkGuideTemplate` (explicit loud warning in `Awake` if unset)
`ChalkGuideSegmentLink.cs`	`[ExecuteAlways] MonoBehaviour`	Keeps one guide `LineRenderer` segment glued between two Point transforms, in Edit mode too.	—
`EvidenceChalkGuide.cs`	plain `MonoBehaviour`	Per-item chalk-tracing ring, built in the item's local space (drags along if grabbed mid-trace). Purely visual/geometric. **Three separately-confirmed-live bugs** documented and fixed in-file: a subscription-ordering race at bootstrap that broke equip/holster visibility; re-running the generic init formula silently overwriting hand-authored guide shapes; a decal-style prop's non-uniform scale collapsing the guide circle to ~1mm radius.	—
`EvidenceTentTool.cs`	`PlayerTool` (EvidenceMarker)	Numbered-tent dispenser; always offers the lowest unused number; translucent aim preview. `TentPickupEnabled = false` **hardcoded** — placed tents can never be picked up/freed; makes `EvidenceTentPickup`/`EvidenceTentPickupCoordinator` (both fully implemented) currently unreachable. Deliberately gives IDENTICAL feedback whether the tent lands on real evidence or a decoy (explicit anti-hint-leakage design — "DO NOT DIFFERENTIATE").	`tentVisuals`

Script	Type	Role	Must-wire fields (no fallback)
`LetterTentTool.cs`	`PlayerTool` (LetterTentMarker)	Separate dispenser for the single letter tent (the body). Split out of `EvidenceTentTool` because a numbered claim (identification among decoys, protected ambiguity) and a letter claim (no pool, no ambiguity) are fundamentally different kinds of claim.	`letterTentStub` (explicit stub placeholder — no real model yet)
`UVRevealable.cs`	`[RequireComponent(EvidenceProp)]`	Hidden-under-normal-light item, visible only under `FlashlightTool`'s UV beam. `LockVisible()` (called on successful dusting) makes it permanently visible; `Remove()` (on successful lift) overrides even that.	—
`HudFollowCamera.cs`	plain `MonoBehaviour`	See Section 11 — the world-space-canvas-follows-camera pattern every HUD uses.	—
`BrushTipReceiver.cs`, `TapeTipReceiver.cs`, `SwabTipReceiver.cs`, `EvidenceBagReceiver.cs`	thin forwarders	Sit on a tool's tip trigger-collider child, forward `OnTrigger*` to the owning tool. `EvidenceBagReceiver` uniquely also checks on `OnTriggerStay` (three insertion conditions can each become true after the item's already in the zone).	their tool reference — no fallback
`ViewfinderFrameMask.cs`	`[RequireComponent(RectTransform)]`	Masks the viewfinder canvas to 4:3 with runtime black bars, recomputed every `LateUpdate`. This session replaced ~20 hand-authored corner-bracket/reticle Image pieces with a single `CameraFrameOverlay` sprite, resized to exactly the visible window every frame so it stays centered through the shutter flash. Also tints a decorative border via `SetConfirmed`.	—
`CameraShutterEffect.cs`, `AimIndicatorUI.cs`, `ViewfinderConfirmationFeedback.cs`, `PhotoCaptureListener.cs`	viewfinder support	Shutter blackout flash; red/green aim dot (deliberately reads `IsConfirmedForCapture`, not the broader `CanCapture`, to avoid revealing "this is evidence" prematurely); bridges confirm state to the frame mask; renders the actual saved photo through a dedicated hidden camera so HUD chrome never bakes in.	—
`ToggleGrab.cs`	`[RequireComponent(Grabbable)]`	Converts hold-to-grab into click-to-grab/click-to-release (sticky select); pressing grip/select again force-releases.	`leftSelectAction`/`rightSelectAction`

Script	Type	Role	Must-wire fields (no fallback)
`OpenableFurniture.cs`, `AtticLadderLever.cs`, `FoldingStairToggle.cs`	scene furniture	Drawer/door proximity-toggle; wall lever + prompt; hinge-rotates an assembly around an explicit world-space pivot (deliberately not reparented — Unity refuses to reparent a nested prefab instance root outside normal drag-and-drop).	`stairToggle` / `target`
`FlashlightTool.cs`	`PlayerTool` (IOC)	Grabbable UV flashlight; forces `LightType.Spot` every `Awake`.	activate actions
`RightHandOnlyFilter.cs`, `LeftHandOnlyFilter.cs`	`IGameObjectFilter`	Physically restrict which hand can grab an interactable (left hand busy holding the bag; right hand shouldn't also grab the bag itself).	—
`EvidenceTentPickup.cs`, `EvidenceTentPickupCoordinator.cs`	pickup + static arbiter	**Currently unreachable** — see `EvidenceTentTool.TentPickupEnabled = false` above. Fully implemented, never instantiated.	—

No literal `// TODO`/`FIXME` tags exist anywhere in this folder — every known-issue marker is a prose comment, several headed in caps ("INTENTIONALLY DISABLED", "CONFIRMED AS A REAL BUG", "AMBIGUOUS CALLS MADE HERE"). Grep for these phrases if you want to find every self-documented rough edge quickly.

7. Script Catalog — UI, Feedback, and VR Input Architecture

7.1 `PlayerUIGate.cs` — the UI mutual-exclusion primitive

```
public static class PlayerUIGate
{
    private static readonly HashSet<object> blockers = new();
    public static bool IsBlocked => blockers.Count > 0;
    public static void Enter(object token) => blockers.Add(token);
    public static void Exit(object token) => blockers.Remove(token);
    public static int LockCount => blockers.Count;
    public static IEnumerable<string> LockOwnerNames { get; }  // diagnostic
}
```

Not a `MonoBehaviour` — a plain static class, zero-wiring to call from anywhere. A `HashSet<object>` of tokens (normally `this`), not a bool/counter, specifically so an unbalanced or duplicate `Enter`/`Exit` (a stale reference across a scene reload, a defensive `OnDisable` exit firing after the normal close path already ran) is always safe — `HashSet.Remove` on a missing token just returns `false`, never throws.

Every real caller (13 confirmed via whole-tree grep): `UtilityMenuController`, `ToolWheelController`, `PhotoAlbumUI`, `EvidenceInventoryUI`, `RecorderReportUI`, `PhotographTool`, `CompletionScreenUI`, `HypothesisCheckpointUI`, `EndingManager` (all pass `this`); `PreSessionFlowController` passes a private plain-`object` field `uiGateToken` — exactly the case `LockOwnerNames`'s reflection fallback exists for. **Pattern**: `Enter(this)` on open, `Exit(this)` both in the normal close path AND defensively in `OnDisable()`; every opener checks `IsBlocked` first.

Chained handoff: `UtilityMenuController.CloseMenu()` calls `Exit(this)` then, in the SAME synchronous call chain (its invoked `MenuEntry.onSelect` UnityEvent), the opened panel's `Show()` calls `Enter(this)` — the gate is briefly unheld between the two, but single-threaded execution means nothing else can run in that gap.

7.2 `HudFollowCamera.cs` — the world-space-canvas-follows-HMD pattern

Every HUD/panel uses a real **World Space** `Canvas` plus this component, because Screen Space - Overlay canvases don't composite correctly under stereo VR rendering (per its own doc comment: "only part of the HUD — usually just center-anchored elements — ends up visible"). Each `LateUpdate`: `transform.SetPositionAndRotation(camera.position + camera.forward * distance + camera.rotation * localOffset, camera.rotation)` — position AND rotation both snap to the camera, so the panel always faces the player squarely. `Distance`/`LocalOffset` are public settable properties, used by every runtime-built panel right after `AddComponent<HudFollowCamera>()`.

Confirmed consumers: `UtilityMenuController` (0.65), `ToolWheelController` (0.65), `RecorderReportUI` (0.75), `EndingManager` (0.75), `CompletionScreenUI`, `HypothesisCheckpointUI`, the (disabled) `DevDiagnosticOverlay`. `PhotoAlbumUI`/`EvidenceInventoryUI` are scene-authored panels — NOT VERIFIED by source grep whether their canvases also carry this component (that would live on a scene/prefab GameObject, not visible to a `.cs` search).

7.3 UI file catalog (`Assets/_Project/Scripts/UI/`, 12 files)

Script	Role
`PlayerUIGate.cs`	See 7.1

Script	Role
`HeldEvidenceStorePrompt.cs`	Ambient "press B to store in inventory" prompt while holding sealed evidence; yields the shared prompt slot to `EvidenceBoxDepositZone`'s more specific prompt when both would apply to the same item. **No** `OnDeviceChange` **re-resolution** (see 7.5 — a plausible latent gap).
`EvidenceInventoryEntryView.cs`	One row in the inventory panel — marker component exposing child references, populated by `Bind(...)`, never builds UI itself.
`EvidenceInventoryUI.cs` (362 lines)	The evidence inventory panel — right stick highlights, right A equips/unequips, right B closes. Registers `OnDeviceChange` re-resolution. Full detail in Section 8.
`EvidenceNotifier.cs`	Presentation-only bridge from `OnEvidenceStatusChanged` to toast text (e.g. "EVD-014 found"). `announceDiscovery` (bool) is a **per-scenario policy field** — `CSI_Environment` sets it false so players can't map the roster by walking around reading toasts. `Marked` is deliberately excluded from the toast-verb switch (see `FeedbackDirector` below — same anti-leak logic).
`NotificationManager.cs` / `NotificationUI.cs`	Scene-wide `Notify`/`ShowPrompt`/`HidePrompt` entry point + the actual toast/prompt popup. Strict two-tier priority: state-driven `Show` messages always win and are never interrupted by an ambient `ShowPrompt`, but the ambient prompt auto-resumes once the queue drains — fixes a documented past bug where `ShowPrompt` could silently swallow every refusal toast.
`PhotoAlbumManager.cs` / `PhotoAlbumUI.cs`	Captured-photo store + gallery panel. **No** `OnDeviceChange` **re-resolution** (gap, see 7.5).
`ReportLineChip.cs`	One report-line card in `RecorderReportUI`; selection driven entirely by thumbstick/button, explicitly NOT uGUI pointer events (see 7.4).
`UIManager.cs`	Flat (non-VR) main-menu scene controller — button-driven, loads `CSI_Environment` by name.
`UtilityMenuController.cs` (435 lines)	Radial menu for utility screens (Notes/Item Logs/Photo Album/Evidence Inventory/...); mechanically identical to `ToolWheelController` but generic (`displayName`/`description`/`UnityEvent onSelect` entries instead of `ToolType`-bound). No shared base class with `ToolWheelController` — both independently reimplement their own `CreateImageChild`/`CreateLabel`/`CreateRingSprite`/`CreateCircleSprite`.
`RecorderReportUI.cs` (742 lines)	Report-writing panel — thumbstick+button chip mover, non-dismissible once open (real procedural step). Explicitly documents abandoning a real drag-and-drop/uGUI pointer implementation: "Meta ISDK ray interactors were confirmed to never reach world-space canvases built by these runtime panels" — and notes this finding held project-wide, not just here. This session shrank its physical scale from `0.001`→`0.0007` (≈50° horizontal FOV instead of ≈67°) to stop it spilling into peripheral vision. **No** `OnDeviceChange` **re-resolution** (gap, see 7.5).

7.4 `Assets/_Project/Scripts/Feedback/` (2 files)

- `FeedbackDirector.cs` — routes `EvidenceStateManager` events to `InteractionFeedback` haptics/tone (the acknowledgement-layer analog of `EvidenceNotifier`'s toast layer). Kept as a SEPARATE component deliberately: "a toast is information, a pulse is acknowledgement." `Marked` is explicitly excluded from `Confirm()` with a "DO NOT ADD IT" comment — acknowledging a mark only on registered evidence would leak correctness through a side channel; `EvidenceTentTool` owns the `Marked` cue instead, unconditionally on both correct and incorrect placements.

- `InteractionFeedback.cs` — haptic pulse + procedural tone (generated via `BuildTone` if no `AudioClip` assigned — the project ships no authored SFX for this). Exactly two channels, semantically opposite: `Confirm` = "the game registered what you did" (fires identically on correct or incorrect actions); `Blocked` = "the game refused to do that" (hard-gated procedural violations only, never used to signal a wrong identification). Haptics fire on BOTH hands unconditionally (most callers don't know which hand performed the action). A per-target scale-pop dictionary prevents a documented compounding-scale bug from rapid repeated pulses in one frame.

7.5 Consolidated controller-input table

All entries VERIFIED via whole-tree grep of `new InputAction(...)` constructor calls; control paths quoted exactly.

Physical control	Exact control path	Scripts reading it
Left X	`<XRController>{LeftHand}/primaryButton`	`UtilityMenuController` (open utility menu), `PhotographTool` (aim toggle)
Left Y	`<XRController>{LeftHand}/secondaryButton`	`ToolWheelController` (open tool wheel)
Right stick	`<XRController>{RightHand}/thumbstick`	`UtilityMenuController`, `ToolWheelController`, `EvidenceInventoryUI`, `PhotoAlbumUI`, `RecorderReportUI`, `HypothesisCheckpointUI`, `EndingManager` (all as Vector2 nav)
Right A	`<XRController>{RightHand}/primaryButton`	`EvidenceInventoryUI` (equip/unequip), `PhotoAlbumUI` (big view), `RecorderReportUI` (confirm), `CompletionScreenUI`, `HypothesisCheckpointUI`, `EndingManager`, `EvidenceBoxDepositZone` (deposit)
Right B	`<XRController>{RightHand}/secondaryButton`	`EvidenceInventoryUI` (close), `HeldEvidenceStorePrompt` (store), `PhotoAlbumUI` (close), `RecorderReportUI` (move chip Pool↔Notes), `AtticLadderLever`, `EvidenceTentPickup`
Right trigger / grip	`triggerPressed`/`gripPressed`	`FingerprintStationTool` only
Left/Right trigger	`trigger`	`OpenableFurniture` only

Every one of these is a separate loose `InputAction` instance, not a shared `InputActionAsset` action — Unity's Input System delivers the same physical value to EVERY currently-enabled one simultaneously. There is **no automatic mutual exclusion at the Input System level**; exclusivity relies entirely on (a) `PlayerUIGate.IsBlocked` checks before opening, and (b) every panel's own `Update()` early-out (`if (!panelRoot.activeSelf) return;` or equivalent) once open. This is a structural convention every new UI script must remember to follow, not something the bindings themselves enforce.

Confirmed hardware-stability gap: `UtilityMenuController`, `ToolWheelController`, and `EvidenceInventoryUI` all register `InputSystem.onDeviceChange` and re-enable their loose actions on `Added`/`Reconnected`, citing a confirmed Quest 3S bug (XR controller devices torn down/recreated mid-OpenXR-session, leaving a loose action's stale resolution pointed at a replaced device — symptom: "right stick does nothing, A does nothing"). `PhotoAlbumUI`**,** `RecorderReportUI`**, and** `HeldEvidenceStorePrompt` **do NOT register this handler**, despite using the identical loose-action pattern on the same physical controls. This is a plausible latent bug (these three could go unresponsive after the same reconnect event the other three explicitly guard against) — confirmed by absence in the read source, not a guess. **If you touch VR input on Quest, add the** `onDeviceChange` **re-resolution to these three.**

7.6 How to add a new controller action

Follow the established pattern exactly — do not use an `InputActionAsset`/`PlayerInput` component, this project doesn't use them for gameplay input:

```
private InputAction myAction;

private void Awake()
{
    myAction = new InputAction("MyFeature_Name", InputActionType.Button, "<XRController>{RightHand}/
secondaryButton");
}
private void OnEnable()
{
    myAction.Enable();
    InputSystem.onDeviceChange += OnDeviceChange;   // do this even if older scripts sometimes skip
it
}
private void OnDisable()
{
    InputSystem.onDeviceChange -= OnDeviceChange;
    myAction.Disable();
}
private void OnDeviceChange(InputDevice device, InputDeviceChange change)
{
    if (change != InputDeviceChange.Added && change != InputDeviceChange.Reconnected) return;
    myAction.Disable();
    myAction.Enable();
}
private void OnDestroy() => myAction?.Dispose();
```

Before picking a physical button, check Section 7.5's table for an existing binding on the same control and confirm your new consumer's active context genuinely never overlaps with the existing ones (or gate it behind `PlayerUIGate` the same way).

8. VR / Meta XR Architecture

Stack (VERIFIED, Section 2.3): OpenXR 1.17.1 + Meta XR SDK 205.0.0 (`com.meta.xr.sdk.all`, pulling core/audio/voice/haptics/mrutilitykit/platform/interaction/interaction.ovr) + XR Interaction Toolkit 3.5.1 + XR Hands 1.9.0 + Input System 1.20.0. Android OpenXR render mode: **Single Pass Instanced**.

Camera rig: `[BuildingBlock] Camera Rig` (root GameObject) — `OVRCameraRig` + `OVRManager` + `OVRHeadsetEmulator` + `PlayerHeightOffset`. This IS the player; there is no separate "Player" hierarchy. `PlayerHeightOffset` shifts the tracking space vertically (Floor Level tracking convention) so in-VR eye height can differ from the wearer's real calibrated height, without affecting grab distances.

Hand/controller interaction: built on `Oculus.Interaction` types (`Grabbable`, `HandGrabInteractable`, `GrabInteractable`, `PointerEvent`) — every `PlayerTool` requires a `Grabbable`. `ToggleGrab` converts Oculus's default hold-to-grab into click-to-grab/click-to-release. `RightHandOnlyFilter`/`LeftHandOnlyFilter` (`IGameObjectFilter`) physically restrict which hand can grab a specific interactable.

Locomotion: `Oculus.Interaction.Locomotion` types (`LocomotionAxisTurnerInteractor`, `SlideLocomotionBroadcaster`), paused/resumed via `LocomotionSuspender`'s reference-counted `Suspend()`/`Resume()` while any exclusive UI is open.

Gameplay input is NOT Unity's `PlayerInput`**/**`InputActionAsset` **component system.** Every controller binding in this project is a raw, code-constructed loose `InputAction` against a literal control path string (`<XRController>{Hand}/control`), enabled in `OnEnable`, disabled in `OnDisable`. See Section 7.5/7.6 for the full table and the exact pattern to follow for new input.

UI interaction is deliberately NOT the Meta ISDK ray → `PointableCanvasModule` → **uGUI pointer pipeline**, even though `Pointable Canvas Module` (`EventSystem` + `PointableCanvasModule` + `BaseInput`) exists in the scene. Multiple independent panels in this codebase tried that pipeline and confirmed it never reliably reached their runtime-built World Space canvases; every panel now uses raw thumbstick/button polling instead (Section 7.3). If you add a new panel, follow that pattern — do not reach for the ray/pointer pipeline expecting it to work, per this project's own confirmed experience. (One exception worth knowing: earlier debugging this session added `PointableCanvas` components to `EvidenceInventoryPanel` and the pairing UI canvas while chasing a black-panel rendering bug that turned out to be caused by a Single-Pass-Instanced/stereo-rendering shader issue unrelated to the pointer pipeline — those `PointableCanvas` components are currently harmless leftovers, not a working ray-input path.)

9. Data Model — ScriptableObjects and Data/ Assets

9.1 Every ScriptableObject type (10 found via whole-tree grep for `: ScriptableObject`)

Type	File	`[CreateAssetMenu]` path	Purpose
`EvidenceDefinition`	`CaseFile/EvidenceDefinition.cs`	`VR-CSI/Evidence Definition`	Per-item authoring record (Section 4.1)
`AssessmentUploadConfiguration`	`Assessment/AssessmentUploadConfiguration.cs`	`VR-CSI/Assessment/Upload Configuration`	Per-environment upload endpoint/retry config (Section 13)
`ScenarioDefinition`	`Assessment/ScenarioDefinition.cs`	`VR-CSI/Assessment/Scenario Definition`	Stable scenario identity for upload/scoring
`EndingSet`	`Ending/EndingSet.cs`	`VR-CSI/Ending/Ending Set`	Submission gate + conclusion options + all 5 authored endings
`ObjectiveSet`	`Objectives/ObjectiveSet.cs`	`VR-CSI/Objectives/Objective Set`	Investigation objectives with unlock/completion/failure trees
`NarrativeEventSet`	`Narrative/NarrativeEventSet.cs`	`VR-CSI/Narrative/Narrative Event Set`	Requirement-gated or manually-fired story beats
`RelationshipDefinition`	`Relationships/RelationshipDefinition.cs`	`VR-CSI/Relationships/Relationship Definition`	One candidate deduction-board connection (one asset per relationship)
`HypothesisCheckpointSet`	`Deduction/HypothesisCheckpointSet.cs`	`VR-CSI/Hypothesis Checkpoint Set`	Mid-scenario forced-theory checkpoints
`RequirementDefinition`	`Requirements/RequirementDefinition.cs`	`VR-CSI/Requirements/Requirement Definition`	Reusable named requirement tree (none authored yet)
`DialogueLinePool`	`STCS/DialogueLinePool.cs`	`VR-CSI/STCS/Dialogue Line Pool`	Shared line-data asset, used by both STCS and Narrative systems

Important correction to a common assumption: `EvidenceRecord` (Section 4.1) is a plain `[Serializable]` runtime class, **not** a ScriptableObject — it cannot exist as a standalone `.asset`. Despite the folder name, `Data/Evidence/*.asset` files (e.g. `EVD-014_KitchenKnife.asset`) ARE `EvidenceDefinition` instances (verified by reading the raw YAML: `m_EditorClassIdentifier: Assembly-CSharp::EvidenceDefinition`). The real split between `Data/Evidence/` and `Data/EvidenceDefinitions/` is **scenario roster**, not type: `Evidence/` holds the CSI_Environment roster (real EVD-xxx/BODY-A ids), `EvidenceDefinitions/Tutorial/` holds the Tutorial roster (TUT-xxx ids) — both folders hold the exact same C# type.

9.2 `Data/` folder inventory (by actual verified type, not folder name)

Folder	Contents
`Data/Assessment/` (new, uncommitted)	3× `AssessmentUploadConfiguration` (Development/QuestDev/Production) + 1× `ScenarioDefinition`
`Data/DialoguePools/`	29 `DialogueLinePool` assets (per-evidence-per-status dialogue) + `Narrative/` subfolder with 9 more (`DialoguePool_Beat1_InitialBriefing.asset` … `Beat9_GlovesConnectWife.asset`)

Folder	Contents
`Data/Ending/`	`Endings_CSI_Environment.asset` (`EndingSet`, 5 endings)
`Data/Evidence/`	6× `EvidenceDefinition` — the real scenario roster (BODY-A, EVD-014, 015, 017, 018, 019)
`Data/Evidence/HeldPlaceholders/` (new, uncommitted)	NOT ScriptableObjects — placeholder PNG icons + prefabs meant to be dragged into `EvidenceDefinition.icon/.heldPrefab`
`Data/EvidenceDefinitions/Tutorial/`	4× `EvidenceDefinition` — the Tutorial roster (TUT-001/002/003/FP-001)
`Data/Narrative/`	`NarrativeEvents_CSI_Environment.asset` (`NarrativeEventSet`)
`Data/Objectives/`	`Objectives_CSI_Environment.asset` (`ObjectiveSet`)
`Data/Relationships/CSI_Environment/`	7× `RelationshipDefinition` (`REL-bottle-supports-brother`, `REL-gloves-blood-matches-victim`, `REL-gloves-fingerprint-identifies-wife`, `REL-hidden-gloves-contradicts-wife`, `REL-knife-consistent-wound`, `REL-physical-contradicts-wife`, `REL-spatter-consistent-reconstruction`)
`Data/Requirements/`	Empty — no authored `RequirementDefinition` assets exist yet
`Data/Roles/`	Empty — no ScriptableObject type in this audit maps here; purpose NOT VERIFIED
`Data/Scenarios/`	`GroundTruth_CSI_Environment.json` (plain JSON export, not a ScriptableObject) + `HypothesisCheckpoints_CSI_Environment.asset` (`HypothesisCheckpointSet`) — folder name is legacy now that scenario identity lives in `Data/Assessment/Scenario_CSI_Environment.asset`

9.3 A concrete worked example — `EVD-014_KitchenKnife.asset` (raw YAML excerpt)

```
m_EditorClassIdentifier: Assembly-CSharp::EvidenceDefinition
evidenceId: EVD-014
displayName: Kitchen knife
relevance: 0                        # EvidenceRelevance.Critical
interactionRadius: 1.5
requiresFingerprinting: 1
fingerprintBeforeCollection: 1
requiresFlipBeforeFingerprinting: 0
reportLineOptions:
- id: knife-blade-length
  text: Blade measures approximately 15 cm...
  isAccurate: 1
  prerequisites: []
- id: knife-wound-match
  text: Blade dimensions are consistent with the sharp-object wound...
  isAccurate: 1
  prerequisites:
  - type: 0                         # PrerequisiteType.ReportLineSelected
    evidenceId: BODY-A
    lineId: body-a-wound-sharp-object
minimumLinesRequired: 3
chalkCircleRadius: 0.22
requiresSwabbing: 1
```

```
collectedBySwab: 0
examinedInPlace: 0
icon: {fileID: 21300000, guid: 1582e4e6fc9adac4eacd81d2ef735f55, type: 3}
heldPrefab: {fileID: -5133974656848836639, guid: 3b330ccd5cb7d044a9a38b2fa8f003f6, type: 3}
```

Note `relevance: 0` and every `prerequisites[*].type: 0` — raw persisted enum ints, exactly the ordinal-drift risk in 9.4.

9.4 Enum order-sensitivity audit — full list

Enum	File	Status
`EvidenceStatus`	`CaseFile/EvidenceStatus.cs`	**Hardened** — explicit ints, in-file "insertion log" of 3 past incidents
`EvidenceRelevance`	`CaseFile/EvidenceRelevance.cs`	**Hardened** — explicit ints
`ToolType`	`RoleSystem/ToolType.cs`	**Hardened** — explicit ints; cited by `EvidenceStatus.cs` as a second confirmed prior incident
`RequirementNodeKind`, `RequirementLogicalOperator`, `RequirementType`	`Requirements/RequirementEnums.cs`	**Hardened** — explicit ints, `RequirementType` doc comment states the discipline explicitly
`SessionEventType`	`Logging/SessionEventType.cs`	Mitigated differently: persisted as a **string** (`.ToString()`), not an int — reordering is safe; only renaming a member is not
`NarrativeTriggerMode`	`Narrative/NarrativeEvent.cs`	**Hardened** — explicit ints
`AssessmentUploadStatus`, `AssessmentUploadErrorCategory`	`Assessment/AssessmentUploadQueueEntry.cs`	**Hardened** — explicit ints, persisted to local JSON queue files
`HypothesisTriggerType`	`Deduction/HypothesisCheckpointSet.cs`	**NOT hardened** — implicit ordinals, but persisted via `HypothesisTrigger.type` in every `HypothesisCheckpointSet` asset. **Latent risk — recommend adding explicit `=0/1` values.**
`PrerequisiteType`	`CaseFile/SelectionPrerequisite.cs`	**NOT hardened** — implicit ordinals, persisted via `SelectionPrerequisite.type` (confirmed `type: 0` in YAML). **Latent risk, same recommendation.**
`EvidenceCustodyLocation`	`CaseFile/EvidenceCustodyLocation.cs`	Explicit ints given, but only used on a runtime (non-persisted-to-.asset) field — NOT VERIFIED whether it's ever serialized to the local JSON upload queue or session log in a way that would matter.

Design pattern worth following for new authoring vocabulary that needs to grow: this project prefers a **free string field over a new enum** where the vocabulary will keep expanding — `RelationshipDefinition.relationshipType` is deliberately a plain string, and its own doc comment names `EvidenceStatus` and `ToolType` as the two prior incidents this choice was made to avoid repeating.

10. Evidence System — full lifecycle (verified against source, not assumed)

```
NotFound → Found → PhotographedRaw → ChalkMarked → Marked → PhotographedFinal
  → Logged → ReadyForCollection → Collected → Sealed → Processed
```

This is `EvidenceStateManager.RequiredSequence` exactly, the one authority for ordering. Four axes are deliberately kept independent — conflating them is the most common conceptual mistake a new contributor can make:

1. **Evidence identity** — `EvidenceDefinition` (authored, ScriptableObject, per scenario item).
2. **Procedure state** — `EvidenceStatus` on `EvidenceRecord`, advanced only through `EvidenceStateManager.Mark*` methods and gated by `RequiredSequence` + per-item blocking rules (swab/fingerprint/on-scene-fingerprint/in-place-examination).
3. **Custody/location** — `EvidenceCustodyLocation` on `EvidenceRecord` (Scene/Inventory/Held/EvidenceBox), owned exclusively by `EvidenceInventory`, completely orthogonal to status. An item can be `Sealed` (status) and still sitting in `Scene` custody until the player actually admits it to inventory.
4. **Fingerprint lab side-track** — `FingerprintLabStatus` (NotSent/SentToLab/Matched), fully decoupled, never interacts with `RequiredSequence`.

Two fingerprint shapes exist, chosen per item via `EvidenceDefinition.fingerprintBeforeCollection`: (a) default — fingerprint after the item is `Sealed`, gates `Processed`; (b) on-scene (currently only the gloves, EVD-019) — fingerprint before logging, gates `Logged` instead, and additionally requires a flip (`requiresFlipBeforeFingerprinting`) which itself requires swabbing done first if `requiresSwabbing` is also set (real-procedure ordering: swab exposed trace before flipping).

What blocks an invalid transition: `EvidenceStateManager.SetStatus` (private, the single funnel every `Mark*` method goes through) checks, in order: unknown id → reject; status not in `RequiredSequence` → reject; `examinedInPlace` item past `Logged` → reject; not exactly the next required step → `Duplicate` (harmless, same-or-backward) or `Violation` (skipped ahead, logged); then the four per-item blocking rules again (defense-in-depth). A `Violation` fires `OnEvidenceTransitionBlocked` and is logged to `SessionEventType.EvidenceTransitionBlocked` — this becomes a `ProceduralViolationDto` row in the uploaded assessment payload (Section 13).

11. Evidence Inventory / Held-Item System (new subsystem — read this before touching custody/equip code)

Components: `EvidenceInventory` (custody + spawn/despawn the held visual), `HeldItemCoordinator` (the single authority for what's held — tools AND evidence), `EvidenceInventoryUI` + `EvidenceInventoryEntryView` (the panel), `EvidenceBoxDepositZone` (terminal deposit), `HeldEvidenceStorePrompt` (ambient store prompt).

How evidence enters inventory: `EvidenceInventory.Add(evidenceId)` — requires the item's `EvidenceRecord` exists and is at-or-past `Sealed`, no existing duplicate entry, capacity not exceeded (0 = unlimited). On success: creates an internal `Entry`, sets `custodyLocation = Inventory`, hides the scene `EvidenceProp` **only on success** (partial-success safety), logs `EvidenceAddedToInventory`, fires `OnCustodyChanged+OnInventoryChanged`. **NOT VERIFIED which tool call-site actually invokes** `Add` — this audit confirmed `EvidenceInventory`'s own implementation but not every caller across the codebase; check `EvidenceBagTool`/`SwabTool`'s seal paths first.

How it becomes held: `HeldItemCoordinator.EquipEvidence(evidenceId, handAnchor)` — holsters the current tool first (`PlayerToolRegistry.HolsterCurrent()`), then `EvidenceInventory.EquipHeld(...)` — requires custody is exactly `Inventory` and `EvidenceDefinition.heldPrefab != null` (refuses and warns otherwise; **never** falls back to showing the raw scene model). Defensively unequips any different currently-held item first. Instantiates/reparents `heldPrefab` under `handAnchor` at the item's authored `heldLocalPosition`/`heldLocalEulerAngles`/`heldLocalScale`.

How tools and evidence coexist: `HeldItemCoordinator.CurrentCategory` (`None`/`Tool`/`Evidence`) is computed live, never cached, by reading `PlayerToolRegistry.CurrentRole` and `EvidenceInventory.CurrentHeldEvidenceId`. Equipping a tool from the wheel unequips held evidence first (deterministically inside `ToggleEquipTool`, AND as a backstop via `PlayerToolRegistry.OnToolGrabbed` in case a tool is physically grabbed bypassing the wheel entirely). Equipping evidence holsters the current tool first. At most one of {a tool, a piece of evidence} is ever in-hand.

How evidence leaves the scene / is deposited: `EvidenceInventory.Deposit(evidenceId, boxId)` — Held→EvidenceBox, terminal, requires `evidenceId == CurrentHeldEvidenceId`. Destroys the spawned visual, removes the entry from the dictionary entirely, sets `custodyLocation = EvidenceBox` (never reverted). `EvidenceBoxDepositZone` re-validates full eligibility on every prompt refresh AND every right-A press (never cached): `PlayerUIGate` not blocked, `HeldItemCoordinator.CurrentCategory == Evidence`, `EvidenceInventory.IsHeld`, not already deposited, custody `Held`, at-or-past `Sealed`, and both the item's own `depositRequirement` and the zone's `additionalRequirement` (both `RequirementNode`, evaluated via `RequirementEvaluator`) satisfied. Entering the trigger alone deposits nothing — an explicit right-A press is required.

Requirements determining eligibility: two independent `RequirementNode` gates stack — one authored per-item (`EvidenceDefinition.depositRequirement`, default vacuous ALL/always-true) and one per-zone (`EvidenceBoxDepositZone.additionalRequirement`, same default). Both must pass.

Logging: `EvidenceAddedToInventory`, `EvidenceEquipped`, `EvidenceUnequipped`, `EvidenceDeposited` (plus `EvidenceDepositAttempted`/`EvidenceDepositBlocked` — presumably fired by `EvidenceBoxDepositZone`, not independently confirmed this pass) — all real `SessionEventType` members.

What happens when a prefab is missing: `EquipHeld` refuses outright and logs a warning if `heldPrefab` is null — it will never silently show nothing or fall back to the raw scene model. `EvidenceDefinition.OnValidate` also warns at author-time if `heldPrefab` is missing (unless `examinedInPlace`).

What happens when inventory operations fail: every public method on `EvidenceInventory`/`HeldItemCoordinator` returns `bool`/void and fails closed with a console warning — nothing throws. Refusal paths (blocked gate, wrong custody, missing prefab) are designed to be safe no-ops from the caller's perspective; the UI layer (`EvidenceInventoryUI`) surfaces a `NotificationManager.Notify` toast + `InteractionFeedback.Blocked()` on a locked-item select attempt.

Pseudocode — the full equip-from-inventory flow:

```
Player opens Evidence Inventory panel (UtilityMenuController → EvidenceInventoryUI.Show)
  → right stick highlights a row (EvidenceInventoryEntryView.Bind data)
  → right A on a highlighted, unlocked row:
      if EvidenceInventory.IsHeld(id):  HeldItemCoordinator.UnequipEvidence()
      else:                              HeldItemCoordinator.EquipEvidence(id, rightHandAnchor)
        → PlayerToolRegistry.HolsterCurrent()
        → EvidenceInventory.EquipHeld(id, handAnchor)
            → refuse if custody != Inventory or heldPrefab == null
            → unequip whatever was previously held
            → instantiate heldPrefab under handAnchor at authored local pose
            → custodyLocation = Held; CurrentHeldEvidenceId = id
            → log EvidenceEquipped; fire OnCustodyChanged + OnInventoryChanged
  → right B closes the panel (never itself unequips anything)
```

12. Requirement System — how to add a new requirement

The evaluator itself is cataloged in Section 5. This section is the practical "how do I add one" guide.

To add a new requirement to an EXISTING leaf type (e.g. gate a new narrative event on an evidence status): in the Unity Inspector, on the owning field (a `RequirementNode`, e.g. `NarrativeEvent.trigger`), set `kind = Leaf`, pick a `requirementType` from the dropdown, and fill in only the field(s) that type reads (see the table in Section 5 — e.g. `EvidenceStatusAtOrAbove` needs `evidenceId` + `requiredStatus`; `RelationshipCompleted` needs only `relationshipId`). Leaving other fields blank is safe — they're simply unread for that type.

To combine multiple conditions: set `kind = Group`, pick `operatorType` (`All`/`Any`/`Not`), and add child `RequirementNode`s to the `children` list (this uses `[SerializeReference]`, so the Inspector shows a "+" to add a polymorphic entry — this is normal, not an error). Remember: empty `All` = true, empty `Any` = false, `Not` requires exactly one child or it fails closed.

To add a genuinely NEW `RequirementType` (a new kind of check nothing above covers): this is a code change, not authoring.

1. Add a new member to the END of `RequirementType` in `Assets/_Project/Scripts/Requirements/RequirementEnums.cs` — **never insert in the middle, never renumber existing members**.
2. Add a corresponding `RequirementFailureReason` member (also append-only).
3. Add an `EvaluateYourNewType(node, visitedAssets)` private method to `RequirementEvaluator.cs`, following the existing shape (missing required id → `MissingXId` failure; no runtime manager → `MissingRuntimeManager`; unknown id → failure; else query the authoritative live-state source and return `Success`/`Failure`).
4. Add a `case RequirementType.YourNewType:` to `EvaluateLeaf`'s switch.
5. Add whatever field(s) your new leaf type needs to `RequirementNode.cs` (it already has a generous set of optional fields — `evidenceId`, `requiredStatus`, `reportLineId`, `narrativeEventId`, `objectiveId`, `relationshipId`, etc. — reuse an existing one if it fits before adding a new field).

Concrete example, from the authored `Endings_CSI_Environment.asset` (Section 9's YAML pattern extended): the "wife convicted" ending's `resolutionRequirement` is an `All` group of 5 leaves — one `ReportLineSelected` (EVD-017, `phone-message-affair`) and four `RelationshipCompleted` leaves (`REL-physical-contradicts-wife`, `REL-gloves-blood-matches-victim`, `REL-gloves-fingerprint-identifies-wife`, `REL-hidden-gloves-contradicts-wife`), plus a `requiredConclusionId: conclusion-wife` gate on the picker. All 5 must pass AND the player must have picked "conclusion-wife" for this ending to be eligible.

Important operational caveat, verified this pass: the four `RelationshipCompleted` leaves above depend on `RelationshipStateManager.CompleteAllSelected()`, whose only known call site (`EvidenceBoardController.SubmitTheory`) sits on a deduction-board component that is **not currently placed in any scene or prefab**. Until the board is wired in (or `CompleteAllSelected` is called some other way), the "wife convicted" ending's relationship leaves may be permanently unsatisfiable in the live game — this is the single highest-priority open engineering question this manual surfaces (see Section 1 and Section 21).

13. Deduction / Story / Ending System

This is the causal chain a new contributor most needs a clear mental model of. Traced directly from source, not assumed.

13.1 The chain, end to end

```
Evidence state change (a Mark* call)
  ■■→ HypothesisCheckpointManager.EvaluateCountThresholds()
  ■      → may enqueue a forced-theory checkpoint (player-visible, logged,
  ■        NOT ending-determining by itself)
  ■■→ RelationshipStateManager.RefreshUnlocks()
  ■      → Locked→Available for any relationship whose unlockRequirement now passes
  ■■→ NarrativeEventManager.RefreshEligibility()
  ■      → may enqueue+play an Automatic-mode beat whose trigger now passes
  ■      → on completion, OnNarrativeEventCompleted may retroactively satisfy
  ■        other requirement trees (objectives, relationships, endings)
  ■■→ ObjectiveStateManager.RefreshAll()
        → advances Hidden→Active→{Completed|Failed}, itself firing narrative
          events on transition, and becoming referenceable elsewhere via
          RequirementType.ObjectiveCompleted

Separately, player presses "Submit Conclusion" (from the Utility Menu):
EndingManager.TrySubmitConclusion()
  → gate-check EndingSet.submissionRequirement (blocks + logs if unmet)
  → HypothesisCheckpointManager.FireSubmissionInitiatedCheckpoints()
      (fires any SubmissionInitiated-type checkpoint unconditionally — purely
       narrative/logging, does NOT feed ending resolution)
  → show conclusion picker (or skip straight through if none authored)
  → FinishSubmission(selectedConclusionId)
      → ResolveEnding(selectedConclusionId):
          filter EndingSet.endings by (requiredConclusionId matches OR blank)
          AND RequirementEvaluator.IsSatisfied(resolutionRequirement);
          pick the highest-priority match, else the one isFallback ending,
          else null
      → CompletionScreenUI.ShowEnding(resolvedEnding)
      → NarrativeEventManager.Fire(ending.followUpNarrativeEventId) if set
      → AssessmentUploadService.QueueSessionForUpload(...) [fire-and-forget]
```

Important, confirmed by reading `CompletionScreenUI.cs`**'s own comment**: hypothesis checkpoints are a parallel, evidence-count-driven "reflection/commitment" mechanic that logs data but is explicitly NOT read by ending resolution. An older version of `CompletionScreenUI` DID key its resolution off the HYP-03 checkpoint's answer directly — that was replaced by the `EndingManager`/`RequirementNode` chain above. Do not reintroduce a direct hypothesis→ending dependency; it was a deliberate architectural change.

13.2 Component-by-component

- `HypothesisCheckpointManager` (singleton on `_Managers`) — subscribes to 3 `EvidenceStateManager` events. `EvaluateCountThresholds()`: for each not-yet-fired `CountThreshold`-type checkpoint, checks `EvidenceStateManager.CountAtOrAbove(thresholdStatus) >= thresholdCount` AND every `isCorrect` reasoning option's prerequisites/extra-requirement are satisfiable, then enqueues it. `FireSubmissionInitiatedCheckpoints()` (called by `EndingManager` as step 1 of Submit Conclusion) fires every not-yet-fired `SubmissionInitiated`-type checkpoint unconditionally — deliberate, since a one-shot player action can't "wait and retry." Answers logged as `SessionEventType.HypothesisSubmitted`.
- `RelationshipStateManager` (singleton) — `RefreshUnlocks()` on the same evidence events (Locked→Available). `MarkSelectedForPair`/`MarkDeselectedForPair` (called from `EvidenceBoardController.TryConnect` — **the board is not in any scene**) set Selected/Available; the physical drag itself has no permission check, even a Locked pair can be drawn. `CompleteAllSelected()` (called once from

`EvidenceBoardController.SubmitTheory`, also currently unreachable) is what actually produces the `Completed` state `RequirementType.RelationshipCompleted` leaves check for.

- `NarrativeEventManager` (singleton) — `RefreshEligibility()` on the same evidence events; `Fire(eventId)` is an unconditional trigger (called by `ObjectiveStateManager` on unlock/completion, and by `EndingManager.FinishSubmission`). `IsCompleted(eventId)` is the authoritative source for `RequirementType.NarrativeEventCompleted`. Deliberately coexists with the older `STCSManager` — STCS stays for simple call-site-driven beats, this system is for requirement-gated/priority/repeatable beats. "STCS" = **Scripted Team Communication System** (explicit, from `STCSManager.cs`'s own doc comment: "simulated non-played teammates commenting on, or completing, scene actions").
- `ObjectiveStateManager` (singleton) — the broadest listener: subscribes to evidence events, `RelationshipStateManager.OnRelationshipStateChanged`, AND `NarrativeEventManager.OnNarrativeEventCompleted`. `RefreshAll()` loops up to 20 passes to resolve same-tick chained unlocks (A unlocking B in the same refresh). Checks `failureRequirement` before `completionRequirement` on an `Active` objective (with an explicit "is this failure condition even configured" guard, `HasRealFailureCondition` — a real past bug where an unconfigured/null `failureRequirement` round-tripped to an always-true empty-ALL group and every objective failed instantly). `IsCompleted(objectiveId)` is the authoritative source for `RequirementType.ObjectiveCompleted`.
- `EndingManager` (singleton) — see the pseudocode above. `ResolveEnding` is a pure function over `EndingSet` + current state; ties in `priority` are broken by authoring order (first found wins, since the comparison is strict `>`, not `>=`).

13.3 A second concrete worked example — the highest-priority ending

`ending-premature-submission` (priority 200, the highest of all 5) resolves when a `Not` group wrapping a single `EvidenceDiscovered` leaf (`EVD-019`, the gloves) passes — i.e. **when the gloves have NOT yet been discovered**. It has a blank `requiredConclusionId` (matches any/no conclusion picked), so `ResolveEnding` will prefer it over every conclusion-specific ending whenever a player submits before finding the gloves and the submission gate (`EndingSet.submissionRequirement` — BODY-A and EVD-014 both at-or-above status ordinal 6) happens to already be satisfied. `ending-inconclusive` (priority 10) is the sole `isFallback` ending, used only if nothing else matches.

13.4 Corkboard / Deduction Board status — dormant, not broken

`EvidenceBoardController`, `DeductionScorer`, `ConnectionLine`, `EvidenceNode` are all fully implemented (drag connections between evidence nodes, `TryConnect→RelationshipStateManager`, `SubmitTheory→CompleteAllSelected+DeductionScorer.Score`) but **confirmed absent from every scene and prefab** in the project (`DeductionScorer.cs`'s own comment states this was confirmed via a full scene/prefab search). This is the single most consequential dormant system in the codebase, since the top-priority ending's relationship-completion requirements depend on it. Whoever picks this up next should either wire the board into `CSI_Environment` or find/author an alternative path to `RelationshipStateManager.CompleteAllSelected()`.

14. Complete Game Flow (chronological, script-level)

1. **Application starts,** `CSI_Environment` **loads.** `SessionLogger.Awake()` (on `_Managers`) starts recording immediately — "before any other script's `Start()` could fire an event it would otherwise miss" (its own doc comment). Generates a fresh `SessionId` (GUID), initializes `LocalJsonLogWriter` (creates `session_<guid>_<timestamp>.jsonl` under `Application.persistentDataPath`), logs `SessionStarted`.
2. **Every other** `_Managers` **component** `Awake`**s** — `EvidenceStateManager` builds its `records` dictionary from `sceneEvidenceDefinitions`; `PlayerToolRegistry.ResetOnPlaySessionStart()` clears static state; `PairingIdentityService`/`AssessmentUploadService` initialize (the latter's `OnEnable` immediately calls `ResumePendingUploads()`, resuming any queue entries left over from a previous run).
3. `RoleSceneLoader.Start()` fires the `"scene_entered"` STCS beat and logs `SceneEntered`.
4. **Pre-session flow.** `PreSessionFlowController.Start()`: if `developmentBypassEnabled` and (`Application.isEditor` OR `Debug.isDebugBuild`) — applies a synthetic identity and unlocks immediately (see the named risk in Section 13/21 — this gate also activates on ordinary Quest Development Builds, not just the Editor). Otherwise: locks investigation (`PlayerUIGate.Enter`, disables locomotion), shows `PairingCodeIdentityPanel`. Student types the pairing code shown on the web dashboard → `PairingIdentityService.ExchangePairingCodeAsync` POSTs it to the backend → on success, `PreSessionFlowController.HandlePairingCompleted` re-verifies pairing and cross-checks the assigned scenario matches `expectedScenario`, then unlocks investigation.
5. **Investigation begins.** Locomotion re-enabled, `PlayerUIGate` cleared. Player can move and open the Tool Wheel (left Y) / Utility Menu (left X).
6. **Tools become available.** `ToolWheelController` reads `PlayerToolRegistry.AllTools` fresh on every open — every `PlayerTool` in the scene shows up automatically.
7. **Evidence interaction.** Walking near a prop within its `interactionRadius` fires `EvidenceProp.OnTriggerEnter` → `EvidenceStateManager.MarkFound`. From there the player runs each item through photograph (raw) → chalk → tent (marked) → photograph (final) → recorder (logged) [→ swab/fingerprint preconditions as configured] → bag+seal (or swab+contain+seal for `collectedBySwab` items).
8. **Evidence inventory.** Once `Sealed`, `EvidenceInventory.Add` admits the item; from the inventory panel the player can `EquipHeld`/`UnequipHeld`; `EvidenceBoxDepositZone` accepts a final deposit.
9. **Reactive systems fire throughout, on every relevant evidence event** — `HypothesisCheckpointManager` (count-threshold checkpoints), `RelationshipStateManager` (unlocks), `NarrativeEventManager` (automatic beats), `ObjectiveStateManager` (objective progression), `EvidenceNotifier`/`FeedbackDirector` (toast/haptic feedback), `SessionLogger` (event log), `ProceduralGateValidator` (in-memory violation log).
10. **Deduction.** (Currently dormant — see 13.4. As designed: connect evidence pairs on a corkboard, `SubmitTheory` completes selected relationships and scores them.)
11. **Hypothesis checkpoints** fire automatically at configured evidence-count thresholds, or unconditionally at Submit-Conclusion time for `SubmissionInitiated`-type ones.
12. **Conclusion.** Player opens the Utility Menu → Submit Conclusion → `EndingManager.TrySubmitConclusion()` (Section 13's pseudocode).
13. **Ending resolution** → `CompletionScreenUI.ShowEnding` displays the resolved ending's text; waits for right-A; loads the `Main` menu scene.
14. **Session logging** has been running continuously since step 1; `SessionLogger.EndSession()` fires on `OnApplicationQuit`/`OnDestroy` (idempotent).
15. **Assessment construction.** `AssessmentSessionBuilder.Build(...)` (called from `EndingManager.FinishSubmission`) reads the just-completed session's log file plus live manager state (evidence records, relationship records, objective counts) into an `AssessmentSessionUploadDto`.

16. **Upload.** `AssessmentUploadService.QueueSessionForUpload` writes the payload locally first (survives a crash/restart), then attempts upload if paired with a valid token; retries with exponential backoff on transient failure; sits at `AwaitingPairing` if the device isn't currently paired as the same student who queued it.
17. **Dashboard processing.** `POST /api/v1/assessment-sessions` validates the bearer token, writes `assessment_sessions` + 7 child tables via a SECURITY DEFINER RPC, then synchronously (errors swallowed, never fails the HTTP response) runs `verifySession()` — the real server-side scoring pipeline, entirely independent of any score Unity itself reported.

15. Logging / Assessment Pipeline (full detail)

15.1 `SessionLogger` and the local event log

`SessionLogger` (singleton on `_Managers`) starts recording in `Awake()`, unconditionally, the instant `_Managers` spins up — never triggered externally. `LogEvent(type, targetId, payload)` builds a `SessionEvent` (`sessionId`, `sequenceNumber` (auto-incrementing), `timestampMs` (session-relative elapsed time via `Time.realtimeSinceStartupAsDouble`, deliberately NOT wall-clock, "so timestamps are comparable across sessions regardless of wall-clock time"), `eventType` (**the enum's** `.ToString()`**, not its int** — deliberate: `JsonUtility` would serialize an enum as a raw int, and a log file on disk "outlives the enum's declaration"; a later reorder would silently corrupt every past log's meaning), `targetId`, `payload`).

Only two events are auto-wired directly inside `SessionLogger.cs` itself (subscribed to `EvidenceStateManager`'s events in `OnEnable`): `EvidenceStatusChanged` and `EvidenceTransitionBlocked`. Every other `SessionEventType` member is logged by its own owning system calling `SessionLogger.Instance.LogEvent(...)` directly (e.g. `EvidenceInventory` logs the four inventory events, `EndingManager` logs the conclusion/ending events, `HypothesisCheckpointManager` logs `HypothesisSubmitted`).

`SessionEventType` **— full 38-member list, in declared order** (a member's declared ORDER doesn't matter for persistence since it's stored as a string, but do not RENAME a member once shipped — old log lines would then contain a string a later reader no longer recognizes):

```
SessionStarted, SessionEnded, SceneEntered, PPEEquipped, SceneLogSigned,
EvidenceStatusChanged, EvidenceTransitionBlocked, ToolEquipped, ToolUnequipped,
PhotoTaken, BriefingStatementViewed, BriefingStatementFlagged, BriefingCompleted,
HypothesisSubmitted, ReportLinesSelected, DeductionLinkCreated,
DeductionLinkRemoved, DeductionSubmitted, NonEvidenceMarked, MarkerReclaimed,
MarkerReclaimBlocked, RelationshipUnlocked, RelationshipAttempted,
RelationshipRemoved, NarrativeEventTriggered, ObjectiveUnlocked,
ObjectiveCompleted, ObjectiveFailed, ConclusionSubmissionAttempted,
ConclusionSubmissionBlocked, ConclusionSelected, EndingSelected,
EvidenceMarkerRejected, EvidenceAddedToInventory, EvidenceEquipped,
EvidenceUnequipped, EvidenceDepositAttempted, EvidenceDepositBlocked,
EvidenceDeposited
```

Its own class doc: "This is the single place a new phase registers a new event type — nothing else in the project should invent its own event-type strings." `AssessmentSessionBuilder`**'s** `ProceduralViolationEventTypes` **set treats exactly four of these as procedural violations**: `EvidenceTransitionBlocked`, `NonEvidenceMarked`, `MarkerReclaimBlocked`, `EvidenceMarkerRejected`.

Local persistence: `LocalJsonLogWriter` (implements `ISessionLogWriter`) — newline-delimited JSON (JSONL), one file per session (`session_<guid>_<timestamp>.jsonl` under `Application.persistentDataPath`), chosen specifically so a crash mid-session leaves every prior line readable (no closing bracket to manage). `Flush()` opens the file, seeks to end, writes, force-flushes to the OS; on `IOException` it rolls back to the pre-write length and sets `LastFlushSucceeded = false` (the retry counters are deliberately NOT reset on a failed flush — "a transient disk failure must not reset the retry thresholds"). `ISessionLogWriter`'s doc comment names a future `SupabaseLogWriter` as the intended second implementation — not built yet; `LocalJsonLogWriter` is the only one today. **Reading it back**: `SessionLogReader.ReadEvents(path)` (in `Assets/_Project/Scripts/Assessment/`, deliberately outside `Logging/` so that folder stays write-only) — never throws on a missing file or a corrupt line (skips with a warning), sorts by `sequenceNumber` before returning.

15.2 Building the assessment payload

`AssessmentSessionBuilder.Build(sessionLogFilePath, config, scenario, selectedConclusionId, resolvedEndingId)` (static, called exactly once, synchronously, at the moment a session ends — while the live

managers are still in-scene, not as an offline tool):

1. `SessionLogReader.ReadEvents` the log.
2. Fill identity/scenario/client-version fields (`studentId`/`classId` from `PairingIdentityService`, both **informational only** — see the security boundary below).
3. `ApplyTimestampsAndDuration` — duration from the `SessionEnded` event's own `timestampMs` if present, else the last event's, "never fabricate a duration from wall-clock now."
4. `BuildEvidenceResults` — one row per `EvidenceProp.All`, read live from `EvidenceStateManager` (final status, swab/fingerprint/flip flags, tent number/letter).
5. `BuildReportLineResults` — replays `ReportLinesSelected` events.
6. `BuildProceduralViolations` — the 4-event-type filter above.
7. `BuildRelationshipResults` — from `RelationshipStateManager.AllRecords`, live.
8. `BuildHypothesisResults` — replays `HypothesisSubmitted` events (last submission per checkpoint wins).
9. `BuildClientReportedScores` — client-side tallies only, explicitly informational (see below).
10. If `config.uploadRawEventLogEnabled`, appends up to `maxEventsPerRequest` raw events (earliest kept, tail dropped if over the cap).

SECURITY BOUNDARY, explicit in source: the DTO deliberately holds NO `isCorrect`/`scoreWeight`/verification fields anywhere — it reports WHAT HAPPENED; scoring is entirely server-side. `studentId`/`classId` on the DTO exist only so the local pending-upload UI can show "queued for `<name>`" — "the server MUST NEVER treat these fields as authoritative identity... it rejects the submission if the token's identity disagrees."

15.3 Upload service — queue, retry, and the restart-authentication design

`AssessmentUploadService` (singleton). `QueueSessionForUpload` builds the DTO, writes it to a local queue (`AssessmentUploadQueueStore` — one index file + one payload-per-session file, every write atomic via a `.tmp`+rename pattern), and — if `automaticUploadEnabled` and the device is currently paired — fires an upload attempt (fire-and-forget; the local save is synchronous and fast, network is not).

`AssessmentUploadStatus` (persisted, ORDER-SENSITIVE): `Pending=0, Uploading=1, Uploaded=2, RetryWaiting=3, Rejected=4, FailedPermanently=5, AwaitingPairing=6`.

Restart-authentication design ("Option B"): the assessment token is never persisted to disk (`PairingIdentityService` holds it in memory only). A queued entry captures `pairedStudentDisplayName`/`pairedClassDisplayName` at queue time; before any real attempt, `IdentityMatchesCurrentPairing` requires the CURRENTLY paired identity to match those exactly (and `TokenIsValidNow`) — a mismatch or unpaired device routes the entry to `AwaitingPairing` with **no network call**, and `PairingIdentityService.OnPaired` triggers an automatic re-scan the moment the right student re-pairs. This prevents one student's queued session ever uploading under a different student's token after a shared-headset restart.

Backoff formula (`ComputeBackoffSeconds`, exact): `initial * 2^(attemptCount-1)`, clamped to `max`; a server `Retry-After` (HTTP 429) is honored ONLY as a floor when larger than the computed value, never to retry sooner. Defaults: `initial=5s`, `max=300s`, `maxRetryAttempts=8`.

Error routing (`HandleResult`): success (or a matching-sessionId duplicate) → `Uploaded`; `AuthenticationFailed` → `AwaitingPairing` (not permanent — resumes on re-pair); `ValidationRejected` → `Rejected` (permanent); `PayloadTooLarge` → `FailedPermanently` (permanent); everything else retryable up to `maxRetryAttempts`, then `FailedPermanently`.

15.4 `AssessmentUploadConfiguration` — the fail-closed gate, and current risk

Three instances exist in `Data/Assessment/`. `AssessmentUploadConfig_Development` (`http://localhost:3000`, `allowLocalhost: true`) is the only one currently functional. `AssessmentUploadConfig_QuestDev` still has an unreplaced placeholder URL; `AssessmentUploadConfig_Production` has an EMPTY `apiBaseUrl`. `IsValidForRuntimeUse(out reason)` is checked before ANY network use by both `AssessmentUploadService` and `PairingIdentityService`, and fails closed on: empty `apiBaseUrl`; localhost without `allowLocalhost`; or (for Production, or any config without `allowLocalhost`) a non-`https://` URL — HTTPS is required even for the Quest-dev config, "certificate validation is never disabled to work around this." **A build pointed at QuestDev or Production today will refuse every pairing/upload attempt until a real HTTPS URL is filled in.**

16. Backend Boundary (Unity ↔ vicsi-dashboard ↔ Supabase)

Unity's responsibility: run the scenario, log events locally, build and upload one assessment payload per completed session, exchange a pairing code for a bearer token. Unity never computes a final score, never stores a standing credential, never talks to Supabase directly.

vicsi-dashboard's responsibility (Next.js): student pairing-code generation, the two endpoints Unity actually calls, server-side scoring (`verifySession`), and instructor-facing results/CSV export. Every Supabase write from a Unity-facing endpoint goes through a SECURITY DEFINER RPC on the service-role client — RLS is the read-path backstop, RPCs are the write-path backstop.

Supabase's responsibility: storage + RLS-scoped reads for the dashboard's own authenticated users (students/instructors); the actual scoring/business logic lives in the Next.js app, not in SQL (the verification RPCs, `014_verification_rpcs.sql`, are pure storage — no scoring math in SQL).

16.1 Pairing: `POST /api/v1/device-pairings/exchange`

Called by `PairingIdentityService.ExchangePairingCodeAsync`. Unauthenticated (the pairing code itself is the credential — hashed with an HMAC secret before touching Postgres, rate-limited per-device and per-IP, and consumed exactly once via the `consume_pairing_code` RPC).

Request:

```
{"payloadVersion":1,"pairingCode":"042917","scenarioId":"CSI-ENVIRONMENT-001","clientVersion":"...",
"deviceInstallationId":"..."}
```

Success response (`{success:true,data:{...}}`):

```
{"assessmentToken":"<shown once>","tokenExpiresAtUtc":"...","assessmentAssignmentId":"...",
 "studentDisplayName":"...","classDisplayName":"...","scenarioId":"...","scenarioVersion":"...",
"scoringRulesVersion":"..."}
```

Error codes: `PAIRING_CODE_INVALID`, `PAIRING_CODE_EXPIRED`, `PAIRING_CODE_USED`, `SCENARIO_MISMATCH`, `ASSIGNMENT_INACTIVE`, `RATE_LIMITED`, generic otherwise.

16.2 Session upload: `POST /api/v1/assessment-sessions`

Called by `AssessmentUploadService`. `Authorization: Bearer <assessmentToken>` from 16.1. Body = the full `AssessmentSessionUploadDto` (Section 15.2's field list). Server pulls only `payloadVersion`/`sessionId` explicitly; the rest is forwarded verbatim to `submit_assessment_session(p_token_hash, p_payload, p_payload_hash)`, which maps it 1:1 into `assessment_sessions` + 7 child tables (snake_case mirrors, per that migration's own header comment). Capped at 2 MiB body / 20,000 events server-side — **this 20,000 figure duplicates `AssessmentUploadConfiguration.maxEventsPerRequest` with no single source of truth; keep them manually in sync if either changes.** A duplicate resubmission (same `sessionId`, same payload hash) is still HTTP 200 with `duplicate:true`; a same-id-different-hash resubmission is `409 SESSION_PAYLOAD_CONFLICT` (permanent — Unity must never retry this as if it were transient).

Success response:

```
{"receiptId":"...","sessionId":"...","uploadStatus":"accepted","duplicate":false,
 "verificationStatus":"verified|requires_review|verification_failed|pending","serverReceivedAtUtc":
"...","payloadVersion":1}
```

16.3 Schema / RLS summary (16 migrations, in order)

`001_profiles.sql` → `002_class_error_log.sql` → `003_require_student_section.sql` → `004_evidence_events_correctness_and_missing_ddl.sql` → `005_rls_policies_conservative_default.sql` → `006_classes_and_enrollments.sql` → `007_scenarios_and_assignments.sql` (creates `students_read_active_enrolled_assignments`: `is_active AND within opens/closes window AND an existing class_enrollments row for auth.uid()`) → `008_assessment_pairing_and_tokens.sql` (hash-only storage, zero client RLS policies by design) → `009_assessment_sessions.sql` (parent + 7 children) → `010_assessment_sessions_rls.sql` (SELECT-only, student-own / instructor-own-class) → `011_assessment_rpcs.sql` (`create_pairing_code`/`consume_pairing_code`/`submit_assessment_session`, service_role only) → `012_rate_limiting.sql` → `013_fix_classes_rls_recursion.sql` → `014_verification_rpcs.sql` (`save_verified_score`/`mark_verification_failed`) → `015_instructor_class_management.sql` → `016_class_assignment_management.sql`.

No secret values are reproduced anywhere in this manual. `.env.example` declares (names only): `NEXT_PUBLIC_SUPABASE_URL`, `NEXT_PUBLIC_SUPABASE_ANON_KEY`, `SUPABASE_SERVICE_ROLE_KEY`, `FORCE_MOCK_DATA`, `PAIRING_CODE_HMAC_SECRET`, `RATE_LIMIT_IP_HASH_SECRET`, plus 8 optional rate-limit override variables.

17. Editing Without Unity UI

For every task below: **SAFE FILE EDIT** = a plain text edit to a `.cs` file that needs no scene wiring. **UNITY SERIALIZED DATA EDIT** = requires editing a `.asset/.unity` YAML file or, more safely, the Unity Inspector/Editor (GUIDs and `fileID`s make hand-editing these files risky — a broken or duplicate GUID reference silently orphans a field). **CODE CHANGE** = a new/modified script, needs Unity to recompile. **EDITOR-ONLY OPERATION** = must be done inside the running Editor (dragging a reference, adding a component) — there is no safe text-file equivalent.

Adding an evidence item — CODE CHANGE: none needed. UNITY SERIALIZED DATA + EDITOR-ONLY.

1. Create a new `EvidenceDefinition` asset: `Assets > Create > VR-CSI > Evidence Definition` (Editor-only). Fill `evidenceId` (must be globally unique, must match whatever id the backend/Supabase will use if this item is ever scored), `displayName`, `relevance`, `interactionRadius`, the fingerprint/swab/flip flags as appropriate, `reportLineOptions`, `chalkCircleRadius`/`chalkPointCount`/`chalkCompletionThreshold`, `icon`, `heldPrefab` + local transform.
2. Place the item's `EvidenceProp` (usually part of a prefab with the standard pickupable-evidence component set — Section 3.3) in `CSI_Environment.unity`, set its `evidenceId` to match, and register the new `EvidenceDefinition` in `EvidenceStateManager.sceneEvidenceDefinitions` (Editor-only, drag into the Inspector list).
3. If it needs an on-scene fingerprint/swab precondition, add an `EvidenceFingerprintMarker`/`EvidenceSwabRequirement` component to the relevant child collider, with `evidenceId` set to the PARENT item's id (these mark a location on a parent item, they are not separate evidence records).
4. If it should be part of an ending/objective/relationship condition, add a `RequirementNode` leaf referencing its `evidenceId` wherever needed (Section 12).
5. Test: `GreyboxFlowTest.RunFullFlow()` (add the new id to its `evidenceIds` array first) drives the full lifecycle without needing VR input.

Adding a tool — CODE CHANGE + UNITY SERIALIZED DATA + EDITOR-ONLY.

1. Append a new member to the END of `ToolType` (`Assets/_Project/Scripts/RoleSystem/ToolType.cs`) — never insert/renumber.
2. Create a new `.cs` class inheriting `PlayerTool`, implement `override ToolType ToolRole => ToolType.YourNewType;`, override `EquipToHand`/`Holster` only if you need the two-handed off-hand-prop pattern (Section 6.1).
3. Build the tool's prefab/GameObject in the scene with a `Grabbable` (required by `PlayerTool`) and whatever trigger colliders + `*TipReceiver` forwarder components your interaction needs, wired to your new script's public handler methods.
4. `PlayerToolRegistry` discovers it automatically — any `PlayerTool` in the scene self-registers in `OnEnable`, and `ToolWheelController` rebuilds its wedges fresh from `PlayerToolRegistry.AllTools` every time it opens. No wheel-side code change needed.
5. Assign a raw `InputAction` for whatever button/stick triggers your tool's action, following the exact pattern in Section 7.6.
6. Equip flow: the wheel calls `HeldItemCoordinator.Instance.ToggleEquipTool(role, handAnchor)` — you don't need to call this yourself, just implement `EquipToHand`/`Holster` correctly.

Changing the evidence procedure order — CODE CHANGE, HIGH RISK.

The order lives in `EvidenceStateManager.RequiredSequence` (`Assets/_Project/Scripts/CaseFile/EvidenceStateManager.cs`), a private `static readonly` array — deliberately NOT the same as `EvidenceStatus`'s own declaration order. **Do not reorder or delete existing** `EvidenceStatus`

members — only append new ones at the end, and if a new step needs to sit BETWEEN existing steps, you must (a) insert the new enum member carefully weighing that every later member's int shifts, (b) update `RequiredSequence` to the new order, and (c) hand-audit every `.asset` that persists a raw `EvidenceStatus` int (`HypothesisCheckpointSet` assets' `thresholdStatus` field, any `RequirementNode.requiredStatus` field) — the source file's own "insertion log" comment documents exactly this pain from three past incidents. This is genuinely the highest-risk change category in the whole codebase; do it deliberately and test with `EvidenceBoundaryTest`/`GreyboxFlowTest`/`StoryContentTest` afterward.

Adding a requirement — see Section 12 in full.

Adding a story event (narrative beat) — UNITY SERIALIZED DATA + EDITOR-ONLY, no code needed for a normal case.

Add an entry to `NarrativeEventSet.events` (`Data/Narrative/NarrativeEvents_CSI_Environment.asset`, Editor-only list edit): `eventId` (unique), `pool` (a `DialogueLinePool` asset — create one via `Assets > Create > VR-CSI > Narrative > Narrative Event Set` context if needed, or reuse an existing pool), `triggerMode` (`Automatic` needs a `trigger` RequirementNode; `Manual` is fired only via `NarrativeEventManager.Instance.Fire(eventId)` from code), `priority`, `repeatable`. `OnValidate` on the asset warns about missing/duplicate ids and a missing pool reference — heed those warnings.

Adding a relationship — UNITY SERIALIZED DATA + EDITOR-ONLY.

Create a new `RelationshipDefinition` asset (`Assets > Create > VR-CSI > Relationships > Relationship Definition`) in `Data/Relationships/CSI_Environment/`: `relationshipId` (unique), `sourceId`/`targetId` (board node ids — either an `EvidenceDefinition.evidenceId` or an arbitrary conclusion-node id), `relationshipType` (free string — do NOT convert this to an enum, see Section 9.4's design-pattern note), `unlockRequirement`, `isCorrect` (scoring only), `scoreWeight`. **Remember**: until the deduction board is wired into the scene (Section 13.4), nothing will ever call `CompleteAllSelected()`, so a new relationship's `RelationshipCompleted` state can currently only ever reach `Available`, never `Completed`, unless you also solve that dormant-system problem.

Adding a hypothesis checkpoint — UNITY SERIALIZED DATA + EDITOR-ONLY.

Add an entry to `HypothesisCheckpointSet.checkpoints` (`Data/Scenarios/HypothesisCheckpoints_CSI_Environment.asset`): `id` (unique), `promptText`, `options`, optionally `reasoningOptions` (each with `isCorrect`, `prerequisites`, `extraRequirement`), and a `trigger` (`CountThreshold` needs `thresholdStatus+thresholdCount`; `SubmissionInitiated` fires unconditionally at Submit Conclusion time).

Adding an ending — UNITY SERIALIZED DATA + EDITOR-ONLY.

Add an entry to `EndingSet.endings` (`Data/Ending/Endings_CSI_Environment.asset`): `endingId` (unique), `priority` (higher wins ties broken by authoring order), `resolutionRequirement` (RequirementNode — see Section 12/13.3 for worked examples), `requiredConclusionId` (optional — blank matches any picked conclusion or none), `endingText` (player-facing), `isFallback` (exactly one ending should have this true — `OnValidate` warns otherwise).

Changing UI without breaking gameplay.

Every panel follows the same skeleton (Section 7): a scene-authored or runtime-built World Space `Canvas` + `HudFollowCamera`, `PlayerUIGate.Enter(this)`/`Exit(this)` around its open/close, `LocomotionSuspender.Suspend()`/`Resume()` while open, raw `InputActions` (never uGUI pointer events) for navigation. If you add a new panel: (1) check `IsBlocked` before opening, (2) register `InputSystem.onDeviceChange` re-resolution (Section 7.6 — three existing panels are missing this, don't repeat the gap), (3) never assume the Meta ISDK ray pipeline will reach your canvas — it's confirmed not to, project-wide.

Changing assessment upload configuration — UNITY SERIALIZED DATA (safe) or CODE CHANGE (for new fields).

Edit the relevant `AssessmentUploadConfiguration` asset directly in the Inspector (`apiBaseUrl`, retry/timeout values) — this is safe, no code change needed. To add a new configuration FIELD, edit `AssessmentUploadConfiguration.cs` and remember its `OnValidate` already checks for duplicate `environmentName`/invalid URLs — extend that check if your new field also needs author-time validation.

Changing scene references — why you (usually) cannot safely text-edit `.unity`/`.asset` files.

Unity serializes object references as `{fileID: N, guid: G, type: T}` triples. `fileID` is a per-file local identifier (stable within one file, meaningless across files), `guid` identifies the target ASSET FILE (via its `.meta` file), and cross-scene/cross-prefab GameObject references are resolved through Unity's own internal PrefabInstance/CorrespondingSourceObject machinery, not a simple lookup. Hand-editing these safely requires knowing the exact `fileID` Unity would have assigned, which you cannot predict — a wrong or duplicate `fileID` produces a **silently broken reference** (null in the Inspector, no error at load time) rather than a parse failure. **Only safe text-editable scene/asset changes**: (a) primitive value fields you can already see the raw value of (a float, a bool, a string) where you're changing the VALUE, not adding/removing a reference; (b) anything you've verified via `grep`/`Read` first and are changing like-for-like. For anything involving adding a new component, a new reference, or a new GameObject, use the Editor (or, as this session repeatedly did, Unity MCP's `execute_code`/`manage_gameobject`/`manage_components` tools, which manipulate the real Editor's object graph correctly rather than raw text).

18. Debugging Without Unity UI

For each problem: symptoms → likely cause → files to inspect → what to verify → safe fix → how to test.

Compilation error

- **Symptoms**: Unity Editor shows a red error banner; MCP `execute_code`/build calls return `"Compilation failed"`.
- **Files**: whatever you just edited. Search terms: the exact error text (Unity's C# compiler errors are standard, greppable `CSxxxx` codes).
- **Verify**: read the file around the reported line number; check for a missing `using`, mismatched brace, or a reference to a type/member that was renamed elsewhere (grep for the old name across `Assets/_Project/Scripts`).
- **Test**: recompile (`Assets > Reimport` / MCP `refresh_unity` with `compile: request`), read the console for `error CS`.

Missing reference / null reference at runtime

- **Symptoms**: `NullReferenceException` in the Console, or a feature silently does nothing.
- **Likely cause**: an unwired "must-wire" serialized field (Section 6.2's table lists every known one) — this project has NO fallback for these by design in several places, so a null field is a real wiring gap, not a bug in the logic.
- **Files/search**: grep the component's class for `[SerializeField]` fields with no `?? fallback`/`FindFirstObjectByType` pattern nearby in `Awake`.
- **Verify**: in the Editor Inspector (or via MCP `execute_code`, `new SerializedObject(component).FindProperty("fieldName").objectReferenceValue`), check the field is actually assigned.
- **Fix**: wire the reference in the Inspector (Editor-only operation — this cannot be done by text-editing the `.unity` file safely, see Section 17). This session's real example: `FingerprintKitTool.brushTipCollider` AND `BrushTipReceiver.fingerprintKitTool` were BOTH unwired, silently breaking the entire fingerprint-dust interaction with no console error at all (a `?.` null-conditional call swallowed it) — always check ALL relevant hops in a forwarder chain, not just the first one.

Evidence won't advance past a step

- **Symptoms**: a `Mark*` call appears to do nothing, or logs a `Violation`.
- **Files**: `EvidenceStateManager.cs` — read `GetItemBlockReason`/the four `IsBlockedBy*` methods.
- **Verify**: check `EvidenceStateManager.Instance.GetRecord(id).status` and the relevant flags (`swabbingDone`/`fingerprintingDone`/`isFlipped`) directly via `execute_code`; cross-check `EvidenceDefinition.requiresSwabbing`/`requiresFingerprinting`/`fingerprintBeforeCollection`/`requiresFlipBeforeFingerprinting` on the item's authored asset.
- **Safe fix**: usually a missing precondition, not a bug — walk the player through the actual required precondition. If it's genuinely a logic bug, `ProceduralGateValidator.GetBlockReason` gives the same human-readable string the player would see.
- **Test**: `EvidenceBoundaryTest.RunAllTests()` covers exactly this class of regression on an isolated rig.

Tool won't equip

- **Symptoms**: wheel wedge is dimmed/inert, or `ToggleEquipTool` silently does nothing.
- **Likely cause**: no `PlayerTool` registered for that `ToolType` in the current scene (dimmed wedge = correct behavior, not a bug); OR `PlayerUIGate.IsBlocked` (something else has the gate); OR (historical, fixed) a stripped-shader `Awake()` throw in `RecorderTool.EnsurePointerLine()` prevented `OnEnable`/registration entirely — device-build-only, never reproducible in Editor Play mode.

- **Verify**: `PlayerToolRegistry.AllTools` (via `execute_code`) — is your `ToolType` present? `PlayerUIGate.LockOwnerNames` — what's currently holding the gate?

Inventory won't open

- **Symptoms**: `EvidenceInventoryUI.Show()` does nothing.
- **Likely cause**: `PlayerUIGate.IsBlocked` (deferential panel, refuses while blocked); `panelRoot` null (should default to `gameObject`, check the Inspector); the historical `Awake`/`Show` deadlock bug (see Section 22 changelog — already fixed, but the failure mode is instructive: a component that starts inactive can never run its own `Awake()`-based fallback, so `panelRoot` must be an explicit Inspector reference, not a runtime-only fallback).

Evidence won't deposit

- **Files**: `EvidenceBoxDepositZone.cs`. Check, in order: `triggerCollider` wired?, `HeldItemCoordinator.CurrentCategory == Evidence`?, item at-or-past `Sealed`?, item's `depositRequirement` AND the zone's `additionalRequirement` both satisfied (`RequirementEvaluator.Evaluate(node, includeChildResults:true)` gives a full failure tree)?

Requirement won't pass

- **Files**: `RequirementEvaluator.cs`. Call `RequirementEvaluator.Evaluate(node, includeChildResults: true)` directly via `execute_code` and read the returned `RequirementEvaluationResult.Reason`/`FailureMessage`/`ChildResults` tree — this gives you the exact failing leaf without guessing. Common causes: an empty `Any` group (always false — probably meant `All`); a `Not` group with 0 or 2+ children (fails closed); a leaf referencing an id that doesn't exist in the live scene (`UnknownEvidenceId`/similar).

Ending won't trigger

- **Files**: `EndingManager.cs`, `EndingSet.cs` asset. Check `submissionRequirement` first (blocks Submit Conclusion entirely if unmet — nothing downstream even runs). Then check each candidate ending's `resolutionRequirement` and `requiredConclusionId` against the player's actual state/pick. Remember: `RelationshipCompleted` leaves depend on the (currently dormant, Section 13.4) deduction board — this is the single most likely real-world cause of "an ending never becomes available."

Session won't log / upload fails

- **Files**: `SessionLogger.cs` (is `Instance` null? check `_Managers` is in the loaded scene), `AssessmentUploadService.cs`/`AssessmentUploadQueueStore.cs`. Check `AssessmentUploadConfiguration.IsValidForRuntimeUse(out reason)` first — a misconfigured `apiBaseUrl` fails closed with no network attempt at all. Check `AssessmentUploadService.GetAllEntries()` for the queue entry's `status`/`lastErrorCategory`/`lastErrorMessage`.

Pairing fails

- **Files**: `PairingIdentityService.cs`. Confirm `configuration.IsValidForRuntimeUse` first. If it's a Development Build behaving oddly, check `PreSessionFlowController.developmentBypassEnabled` — this session's real, repeated example: a Release build correctly requires real pairing (by design), but the SAME `developmentBypassEnabled=true` toggle is silently ignored unless the build is also a "Development Build" (`Debug.isDebugBuild`) — check `EditorUserBuildSettings.development`/the Build Settings checkbox, not just the component field.

UI won't open

- Check `PlayerUIGate.LockCount`/`LockOwnerNames` first, always — this is the single most common cause across every panel in this project.

Controller input doesn't work

- Check Section 7.5's binding table for a conflict on the same physical control; check whether the script registers `InputSystem.onDeviceChange` re-resolution (three panels are known NOT to — Section 7.5); confirm the action was actually `.Enable()`d in `OnEnable` (a common typo is enabling in `Awake` before the device is ready).

Useful commands: `git log --oneline -20` (recent history), `git diff --stat` (what's currently uncommitted), `grep -rn "pattern" Assets/_Project/Scripts` (fastest way to find every caller of anything in this codebase — this manual was built almost entirely this way). Unity batch-mode (`Unity.exe -batchmode -quit -projectPath ... -executeMethod X`) was not used/verified in this pass; prefer the Unity MCP tools or the interactive Editor for anything beyond a pure command-line build.

19. Testing

All 12 files in `Assets/_Project/Scripts/Testing/` are **manually-invoked** `MonoBehaviour` **diagnostic scripts**, not Unity Test Framework `[Test]`/`[UnityTest]` NUnit tests — each exposes a public `RunAllTests()`/`RunFullFlow()`/`RunAndLog()` entry point, run via `FindFirstObjectByType<X>().RunAllTests()` (Editor Play mode, console, or Unity MCP `execute_code`). All are Play-Mode-only (they read/mutate live scene singletons).

File	What it tests	Known limitation (from its own comments)
`AssessmentUploadServiceTest.cs`	DTO round-trip, upload success/duplicate/error-code handling, retry/backoff exhaustion, queue corruption recovery, restart-recovery, the full restart-authentication design	Must neutralize the real scene `SessionLogger.Instance` and `PairingIdentityService.Instance` for the test duration — confirmed as a REAL bug otherwise (every synthetic session would resolve to the same real GUID)
`PreSessionFlowControllerTest.cs`	Pairing gating state machine, via reflection on `HandlePairingCompleted`/`HandlePairingCancelled`	Reuses the real scene `PairingUICanvas`/`PairingCodeIdentityPanel` (no fake substitute possible); **skips entirely** if not found in the loaded scene
`SessionPersistenceTest.cs`	Real JSONL disk persistence, flush timing, `EndSession` idempotency, storage-failure recovery, path-traversal rejection	Real filesystem I/O — creates/deletes actual directories to force `IOException`
`EndingManagerTest.cs`	`ResolveEnding` priority/fallback/conclusion-gating, submission gate, full picker flow	Must force-restore `EndingManager.Instance`/`NarrativeEventManager.Instance` — its own comment states this exact bug PREVIOUSLY BROKE the real scene singleton the first time this test ran (not hypothetical)
`EvidenceBoundaryTest.cs`	State-machine boundary regressions on an isolated rig (null/unknown ids, invalid transitions, tent rules, swab/fingerprint gating, terminal states)	Fully isolated (own `EvidenceStateManager` on a deactivated GameObject); "never advances scene evidence"
`EvidenceInventoryTest.cs`	Inventory/held-item/deposit feature (admission, equip, mutual exclusion, deposit gating)	Must evict/restore the real `PlayerToolRegistry`'s Photographer slot AND the real `PlayerUIGate` blockers set (process-wide static state)
`GreyboxFlowTest.cs`	Full evidence lifecycle end-to-end, Scene Entry through Hypothesis Checkpoint (largest file, ~56KB)	**Explicit disclaimer, verbatim**: "does not simulate VR input... proves the procedural sequence, gates, and logging — not that a controller trigger is bound correctly. A headset pass is still required for that, and this script is not a substitute for one."
`NarrativeEventManagerTest.cs`	Automatic/manual triggers, priority ordering, repeatable flag, `NarrativeEventCompleted` requirement integration	Must force-restore `NarrativeEventManager.Instance` — same clobber-risk pattern
`NotificationPriorityTest.cs`	State-message vs. ambient-prompt priority, resumption behavior	Exists specifically because the original bug was SILENT (a refusal toast enqueued but never drawn, no error anywhere) — samples the real `NotificationUI` label text, not internal state, "what did the player actually see is the whole question"
`ObjectiveStateManagerTest.cs`	Unlock/completion/failure precedence, chained same-tick dependencies, terminal-state stability	Must force-restore THREE singletons (Objective/Relationship/Narrative)

File	What it tests	Known limitation (from its own comments)
`RelationshipStateManagerTest.cs`	Unlock gating, pair lookup, select/deselect semantics, `CompleteAllSelected`, scorer math	Same restore discipline (one instance)
`RequirementEvaluatorTest.cs`	Full AND/OR/NOT truth table, nested trees, circular vs. diamond asset references, legacy backward-compat	Read-only against `EvidenceStateManager` — "it never calls a Mark* method, so running this cannot advance or corrupt evidence state"
`StoryContentTest.cs`	The AUTHORED content end-to-end, against REAL scene singletons (deliberately not isolated) — runs `GreyboxFlowTest` first, then drives narrative beats, 7 objectives, and 4 of 5 endings live	Explicitly states only 4/5 endings are exercised live — `ending-premature-submission` needs pre-gloves-discovery state the session can no longer represent by that point, so it's checked "by construction" instead of via a live resolve
`DevDiagnosticOverlay.cs`	Not a test — an always-on (when enabled) in-headset diagnostic HUD for a tool-wheel/utility-menu button investigation	**Currently disabled** (`Enabled = false` constant, this session's change) — was cluttering normal testing; flip back to `true` if needed

Cross-cutting pattern (important if you write a new test): nearly every test that instantiates a throwaway singleton MUST capture the real scene `Instance` before running and restore it via reflection in a `finally` block, because that type's own `Awake`/`OnDestroy` pattern (`Instance = this` / `if (Instance == this) Instance = null`) will otherwise silently clobber and then PERMANENTLY NULL OUT the real scene singleton the moment your temp object is destroyed. This is not defensive paranoia — multiple test files' own comments confirm it happened for real during development.

Why a passing test doesn't mean "verified on Quest": every test above except the device-build-only historical bugs (Recorder shader-strip) runs in the Editor against a state-machine or isolated rig — none of them drive an actual XR controller through Unity's Input System, and `GreyboxFlowTest`'s own doc comment says so explicitly. A green test suite proves the procedural logic; it does not prove a physical trigger press, a collider overlap, or an OpenXR device-reconnect event behaves correctly. Several real bugs fixed this session (the FingerprintKit unwired collider, the black-panel stereo-rendering issue) were invisible to every test above and only surfaced on a real headset pass.

20. Git / Change Management

Do not perform any Git mutation based on this manual alone — the instructions this manual was written under explicitly forbid staging/committing/pushing as part of producing it. Everything below is guidance for a human (or a future session explicitly asked to act).

Current state (Section 1 has the full detail): branch `master` at `0ab53f2`, 59 changed/untracked paths, none committed. This is real, valuable, working code — the Assessment subsystem was proven this session to actually complete a real network round-trip against the live backend, and the Inventory subsystem's wiring bugs were found and fixed.

Recommended commit grouping (a recommendation, not an instruction executed by this manual):

1. The Assessment/pairing/upload subsystem (`Assets/_Project/Scripts/Assessment/` + `Data/Assessment/` + the `SessionEventType`/`SessionLogger`/`EndingManager` hooks that integrate with it) as one commit.
2. The Evidence Inventory/Custody/HeldItem subsystem (`EvidenceCustodyLocation.cs`, `EvidenceInventory.cs`, `HeldItemCoordinator.cs`, `EvidenceInventoryUI.cs`, `EvidenceInventoryEntryView.cs`, `HeldEvidenceStorePrompt.cs`, `EvidenceBoxDepositZone.cs`, plus their scene wiring) as a second commit.
3. This session's independent bug fixes (`FingerprintKitTool`/`BrushTipReceiver` wiring, `ViewfinderFrameMask`/camera-frame overlay, `RecorderReportUI` panel resize, STCS/Notification `HudFollowCamera` offset fix, `PlayerUIGate`/`UtilityMenuController`/`ToolWheelController` changes) as one or more separate, reviewable commits — don't bundle unrelated fixes with either subsystem above.
4. `DevAgentSettings.asset` / `ProjectSettings/*` changes reviewed SEPARATELY — the flagged `accessToken` value (Section 2.3) may be local-machine-specific and might not belong in a shared commit at all.

What should NOT be mixed: any of the above with a casual "while I was in there" unrelated change — this codebase's own comments show real pain from under-documented restructures (`VR-CSI_SESSION_HANDOFF.md §7i` explicitly flags one).

How to inspect changes: `git status --porcelain` (full path list), `git diff --stat` (line-count summary per file), `git diff -- <path>` (a specific file's diff), `git log --oneline -20` (recent history), `git show --stat HEAD` (what the last commit actually touched).

How to create a safe branch before a risky change: `git checkout -b <descriptive-name>` from a clean-enough starting point — given the current uncommitted state, commit the four groups above FIRST, then branch for new work, so a bad experiment can be abandoned without losing the baseline.

How to recover from a bad change: `git diff` before anything destructive; `git stash -u` (includes untracked files) if you need to temporarily set aside work-in-progress without losing it; never `git checkout .`/`git reset --hard`/`git clean -f` without first confirming (via `git status`) exactly what would be discarded — given the current 59-path uncommitted state, any of those three commands right now would destroy both new subsystems and every fix from this session.

21. "I Need to Change X → Go Here" Quick Reference

I want to change...	Start here	Related files	Risk
Evidence procedure order	`EvidenceStateManager.RequiredSequence`	`EvidenceStatus.cs`, every `.asset` with a persisted `EvidenceStatus` int	**HIGH** — do not reorder existing members, see Section 17
Add/edit an evidence item	`Data/Evidence/*.asset` (new asset), `EvidenceStateManager.sceneEvidenceDefinitions`	`EvidenceDefinition.cs`, `EvidenceProp.cs`, `EvidenceFingerprintMarker.cs`/`EvidenceSwabRequirement.cs`	Low (additive); moderate if editing an EXISTING item's `evidenceId` (breaks every reference to it)
Add a tool	`ToolType.cs` (append), new `PlayerTool` subclass	`PlayerToolRegistry.cs`, `ToolWheelController.cs` (no change needed — auto-discovers)	Low
Inventory / custody	`EvidenceInventory.cs`, `HeldItemCoordinator.cs`	`EvidenceInventoryUI.cs`, `EvidenceBoxDepositZone.cs`, `EvidenceCustodyLocation.cs`	Moderate — new subsystem, less battle-tested
UI panel behavior/layout	The specific panel's `.cs` (`UI/` folder)	`PlayerUIGate.cs`, `HudFollowCamera.cs` — see Section 7 pattern	Low if you follow the existing pattern; moderate if you deviate (e.g. try the uGUI pointer pipeline — confirmed not to work here)
Controller input binding	The consuming script's `InputAction` constructor	Section 7.5's full binding table — check for a conflict first	Low–moderate (silent conflicts possible)
Story / narrative beats	`Data/Narrative/NarrativeEvents_CSI_Environment.asset`	`NarrativeEventManager.cs`, `STCSManager.cs` (older, simpler mechanism)	Low
Objectives	`Data/Objectives/Objectives_CSI_Environment.asset`	`ObjectiveStateManager.cs`	Low
Relationships / deduction	`Data/Relationships/CSI_Environment/*.asset`	`RelationshipStateManager.cs`, `EvidenceBoardController.cs` (**dormant — not in any scene**)	Moderate — see Section 13.4, the board must be wired in for `Completed` state to ever be reachable
Endings	`Data/Ending/Endings_CSI_Environment.asset`	`EndingManager.cs`, `RequirementNode`/`RequirementEvaluator`	Low–moderate (a badly-authored `resolutionRequirement` can make an ending unreachable, not crash anything)
Requirement logic itself	`RequirementEvaluator.cs`, `RequirementEnums.cs`	`RequirementNode.cs`	**HIGH** if changing existing enum semantics; low if only appending a new `RequirementType`
Scoring	`vicsi-dashboard/src/lib/scoring/*.ts` (separate repo)	`AssessmentSessionBuilder.cs` (what data Unity sends), `011_assessment_rpcs.sql`/`014_verification_rpcs.sql`	Moderate — server-side, no Unity redeploy needed, but changes the ground truth students are scored against

I want to change...	Start here	Related files	Risk
Assessment upload config	`Data/Assessment/AssessmentUploadConfig_*.asset` (Inspector edit)	`AssessmentUploadConfiguration.cs`, `AssessmentUploadService.cs`	Low (value edit) / **HIGH** if you weaken `IsValidForRuntimeUse`'s HTTPS-only rule
Pairing flow	`PairingIdentityService.cs`, `PreSessionFlowController.cs`	`PairingCodeIdentityPanel.cs`, backend `device-pairings/exchange/route.ts`	Moderate — see the named development-bypass risk, Section 21's matrix below
Session logging	`SessionLogger.cs`, `SessionEventType.cs`	`AssessmentSessionBuilder.cs` (what gets read back out)	Low to append a new event type; **do not rename an existing member** once real log files reference it

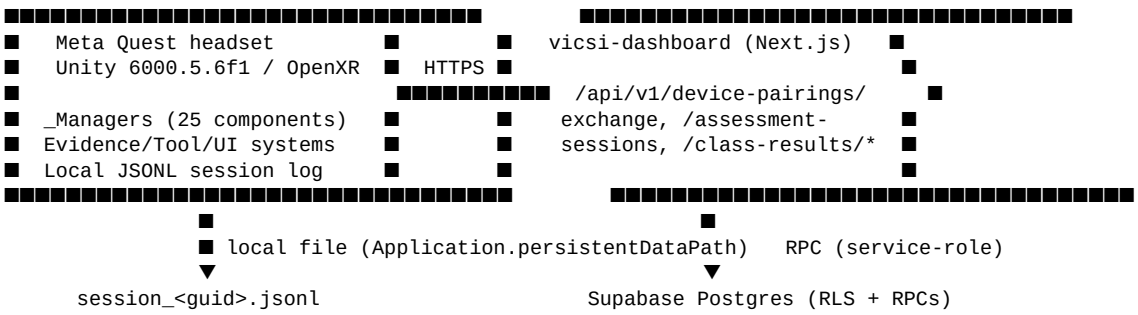
22. Safe Modification Matrix

Component	Classification	Why
`EvidenceStatus` enum	**DO NOT MODIFY CASUALLY**	Persisted as raw int in `.asset` YAML across the whole project; 3 documented past incidents from reordering
`ToolType` enum	**DO NOT MODIFY CASUALLY**	Same class of risk, explicitly cited as a second confirmed incident
`RequirementType`/`RequirementNodeKind`/`RequirementLogicalOperator` enums	**DO NOT MODIFY CASUALLY**	Persisted in every `RequirementNode` across every content asset
`EvidenceStateManager.RequiredSequence`	**HIGH RISK**	The one ordering authority; touching it without the enum-audit discipline in Section 17 will desync from persisted data
`HypothesisTriggerType`, `PrerequisiteType` enums	**CAUTION**	Persisted but NOT explicitly hardened with int values yet — safest fix is to add explicit `=0/1` values before this becomes a fourth incident, not to reorder them
`CSI_Environment.unity` (the scene file)	**HIGH RISK to hand-edit; SAFE via Editor/MCP**	9,089-line uncommitted diff already; opaque `fileID` references make manual text edits dangerous (silent null, not a parse error) — see Section 17
`_Managers` GameObject	**DO NOT MODIFY CASUALLY**	Holds 25 manager singletons as sibling components; deleting/duplicating/deactivating it breaks nearly everything simultaneously
Singleton `Instance` patterns generally	**CAUTION**	Safe to read; unsafe to instantiate a second one without the save/restore-in-`finally` discipline every Testing-folder script uses (Section 19)
`SessionEventType` enum	**SAFE to append; DO NOT rename**	Persisted as a string, not an int, so reordering is safe — but a rename orphans every past log line that used the old name
`AssessmentSessionUploadDto` and nested DTOs	**HIGH RISK to change field meaning/type; SAFE to append a new payloadVersion**	Explicit wire contract with the backend; "never edit an existing field's meaning or type once payloadVersion has shipped" is the project's own stated rule
`AssessmentUploadConfiguration.IsValidForRuntimeUse`'s HTTPS/localhost checks	**DO NOT WEAKEN**	The only thing preventing a misconfigured build from silently uploading real student data to the wrong host, or over plaintext HTTP
`PreSessionFlowController.developmentBypassEnabled`	**HIGH RISK if left true**	Activates on ANY `Debug.isDebugBuild`, not just the Editor — a routine Quest QA sideload build could skip real pairing entirely; always verify this is `false` before any build leaves your machine
Adding a new `EvidenceDefinition`/`RelationshipDefinition`/`NarrativeEvent`/`Objective`/`EndingDefinition` asset	**SAFE**	Purely additive content authoring, each type's own `OnValidate` catches the common mistakes (duplicate/missing ids)
Adding a new `RequirementType` (append-only)	**SAFE**	The pattern is designed for exactly this

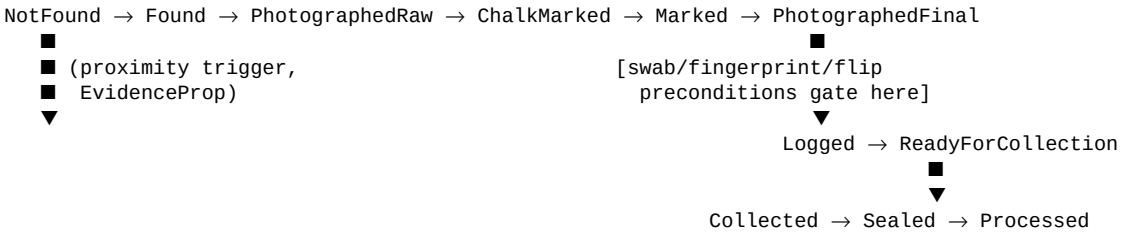
Component	Classification	Why
UI panel visual/layout tweaks (colors, sizes, `HudFollowCamera.Distance`/`LocalOffset`)	**SAFE**	No gameplay coupling
`PlayerUIGate`/`HudFollowCamera` themselves	**CAUTION**	Both are depended on by every exclusive panel in the project; a change here has wide blast radius even though the classes themselves are small

23. Dependency Diagrams

23.1 High-level architecture



23.2 Evidence lifecycle



Orthogonal axis (never conflate): `EvidenceCustodyLocation` — `Scene → Inventory → Held → EvidenceBox`, owned by `EvidenceInventory`, independent of the above.

23.3 Requirement evaluation flow

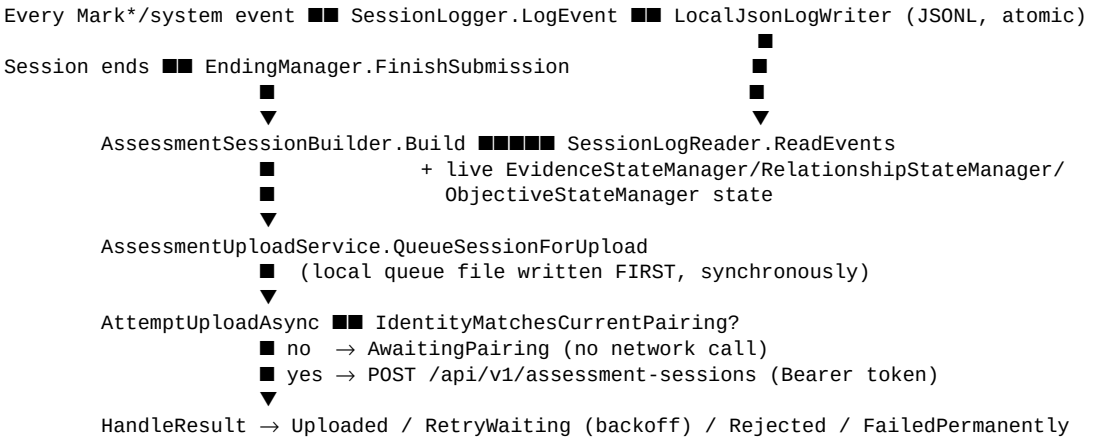
```
RequirementNode (Leaf or Group)
   Leaf  → dispatch on RequirementType → query the ONE authoritative live-state
           source for that type (EvidenceStateManager / RelationshipStateManager /
           ObjectiveStateManager / NarrativeEventManager / SelectionPrerequisite)
   Group → All: short-circuit on first failure (empty = true)
           Any: short-circuit on first pass  (empty = false)
           Not: exactly 1 child, inverted (else fails closed)
```

23.4 Deduction / ending flow

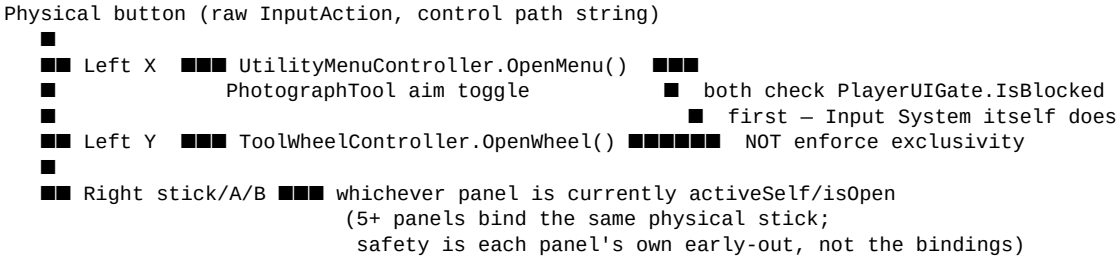
```
Evidence event ■■■■ HypothesisCheckpointManager  (forced-theory, logged, NOT ending-determining)
               ■■■ RelationshipStateManager      (Locked→Available; Completed only via
               ■                                  the currently-unplaced deduction board)
               ■■■ NarrativeEventManager         (Automatic beats; may retroactively
               ■                                  satisfy other requirement trees)
               ■■■ ObjectiveStateManager         (Hidden→Active→{Completed|Failed})

Submit Conclusion ■■ EndingManager.TrySubmitConclusion
                     gate: EndingSet.submissionRequirement
                     → fire SubmissionInitiated checkpoints
                     → conclusion picker (or skip)
                     → ResolveEnding(conclusionId): filter by requiredConclusionId +
                       resolutionRequirement, pick highest priority, else fallback
                     → CompletionScreenUI.ShowEnding
```

23.5 Session logging → assessment upload flow



23.6 Input → Tool/UI flow



24. Glossary

Term	Meaning
EvidenceStateManager	The core singleton owning every evidence item's `EvidenceStatus`, the procedural gate all `Mark*` transitions funnel through.
EvidenceDefinition	The authored ScriptableObject describing one evidence item's identity, requirements, and held-in-hand visual.
EvidenceRecord	The runtime (non-persisted-as-asset) mutable state for one evidence item — status, custody, flags.
Custody / EvidenceCustodyLocation	WHERE an item physically is (Scene/Inventory/Held/EvidenceBox) — independent of its procedural status.
RequirementNode	One node (Leaf or Group) in an AND/OR/NOT condition tree, the shared evaluation mechanism for relationships, objectives, narrative triggers, submission gating, and ending resolution.
STCS	Scripted Team Communication System — simulated non-played teammates commenting on/completing scene actions (explicit, from `STCSManager.cs`'s own doc comment).
HypothesisCheckpoint	A mid-scenario forced-theory panel, triggered by an evidence-count threshold or unconditionally at submission time — logged but NOT read by ending resolution.
SessionEvent / SessionEventType	The atomic logged fact and its 38-member closed vocabulary, written to a local JSONL file per session.
Assessment (session)	The built, uploaded record of one completed play session — evidence results, relationship results, hypothesis answers, procedural violations, optionally the full raw event log.
Pairing	The process of associating a headset session with a real student identity via a short-lived code typed into `PairingCodeIdentityPanel`, exchanged for a bearer token.
PlayerUIGate	The static mutual-exclusion flag every exclusive UI panel checks before opening and holds while open.
HeldItemCoordinator	The single authority for "what's in the player's hand right now" across both tools and evidence — the current correct integration point for new equip logic.
PlayerTool / ToolType	The base class every equippable capability inherits, and the enum tagging which one is currently equipped. There is one player, many tools — not one character per role.
RequiredSequence	`EvidenceStateManager`'s private array — the ONE ordering authority for the evidence lifecycle, deliberately independent of `EvidenceStatus`'s own declaration order.
Deduction Board / EvidenceBoardController	The corkboard-style relationship-connecting minigame — fully implemented but not currently placed in any scene (dormant).
FingerprintLabStatus	A fully decoupled side-track (NotSent/SentToLab/Matched) tracking print IDENTIFICATION, separate from the main evidence lifecycle.
`_Managers`	The single GameObject holding ~25 manager singletons as sibling components — the backbone of the entire scene.

25. Beginner Learning Path

Dependency order, not a time estimate.

1. **Repository structure** (Section 2) — know where things live before reading any code.
2. **Scene/GameObject architecture** (Section 3) — understand that `_Managers` is one GameObject with 25 components before anything else makes sense.
3. `EvidenceStateManager` (Section 4.1/10) — the load-bearing class. Read the real source file, not just this summary, once you've read the summary.
4. `PlayerTool`/`ToolType`/`PlayerToolRegistry`/`HeldItemCoordinator` (Section 4.2/6) — how tools get equipped and how evidence and tools share "what's in hand."
5. **Requirements** (Section 5/12) — the shared evaluator underneath everything from here on.
6. **Evidence Inventory/Custody** (Section 11) — the newest subsystem, builds directly on 3 and 4.
7. **Deduction/Story/Ending chain** (Section 13) — the payoff: how evidence state eventually becomes a player-facing ending. Pay special attention to the dormant-deduction-board caveat.
8. **UI/Input architecture** (Section 7/8) — `PlayerUIGate`, `HudFollowCamera`, the raw-InputAction convention, and why this project doesn't use Unity's default UI event pipeline.
9. **Logging/Assessment/Backend boundary** (Section 15/16) — how a played session becomes a scored record on the server, and exactly where the Unity/server line is drawn.
10. **Make a small safe modification** — Section 21's "SAFE" row items are good first tasks (a new evidence item, a new narrative beat).
11. **Make a complete feature modification** — something spanning at least two systems (e.g. a new tool that interacts with a new requirement type) once 1–9 are genuinely internalized, not skimmed.

26. Documentation Quality Self-Check

- [x] Every important folder documented (Section 2)
- [x] Every important script documented — all 137 files across the 18 script subfolders were read in full by this audit's parallel research passes; the ones not given individual per-method treatment in this manual (e.g. static-environment-art support scripts, `UIManager.cs`) are covered at catalog-table level, consistent with their actual importance to gameplay
- [x] Important GameObjects documented (Section 3)
- [x] Scene hierarchy documented (Section 3, captured live from the running Editor)
- [x] Data structures documented (Section 9)
- [x] Evidence lifecycle documented (Section 10)
- [x] Inventory documented (Section 11)
- [x] Requirement system documented (Section 5/12)
- [x] Story/ending documented (Section 13)
- [x] UI documented (Section 7)
- [x] VR/input documented (Section 8)
- [x] Logging documented (Section 15.1)
- [x] Assessment documented (Section 15.2–15.4)
- [x] Backend boundary documented (Section 16) — no secrets included
- [x] Editing without Unity UI documented (Section 17)
- [x] Debugging documented (Section 18)
- [x] Testing documented (Section 19)
- [x] Git workflow documented (Section 20) — no Git mutations performed while producing this manual
- [x] Dependency diagrams included (Section 23)
- [x] Safe modification matrix included (Section 22)
- [x] Glossary included (Section 24)
- [x] Beginner learning path included (Section 25)
- [x] Current snapshot included (Section 1)
- [x] Unverified information clearly marked (VERIFIED/INFERRED/NOT VERIFIED used throughout, especially Sections 2.3, 4.1's consumer table, 6, 7.3, 9.4)
- [x] No secrets/credentials included — the one flagged item (`DevAgentSettings.asset`'s `accessToken`) is named as a location to review, its value was never read into or reproduced in this manual

Honest gaps, stated rather than hidden: this audit did not open every single one of the 137 scripts personally in the final synthesis pass — it relied on nine parallel research passes, each of which DID read its assigned files in full and cited exact paths/line numbers, and this document was assembled from their verified findings. A small number of files outside the audited folders (e.g. `CompletionScreenUI.cs`'s and `HypothesisCheckpointUI.cs`'s FULL internals, individual scoring-formula files on the backend) are cited by name and role but were not opened in full — every such case is marked NOT VERIFIED or INFERRED at the point it's mentioned rather than asserted as fact. The `Tools/EvidenceScorer` .NET CLI mentioned in `Tools/Audit/scorer.test.mjs` was not independently investigated. `Data/Roles/`'s purpose could not be determined from the repository.

27. Appendix — This Session's Changes (for context, not exhaustive project history)

The uncommitted diff in Section 1 includes, among other things, fixes made during the same session this manual was produced in: wiring two previously-null serialized references in `FingerprintKitTool`/`BrushTipReceiver` that silently broke the entire dust-load/dust-apply gesture; replacing ~20 hand-authored viewfinder corner-bracket/reticle UI pieces with a single `CameraFrameOverlay` sprite in `ViewfinderFrameMask`; shrinking `RecorderReportUI`'s physical panel scale from 0.001→0.0007; fixing an overlapping-`HudFollowCamera`-offset bug between `NotificationCanvas` and `STCSNotificationCanvas`; disabling the always-on `DevDiagnosticOverlay`; and diagnosing (not yet fully resolved) a black-panel rendering issue on the pairing UI canvas under Quest Link's Single Pass Instanced stereo rendering. None of this is committed — see Section 20.

End of manual. This is a snapshot as of 2026-09-19 and will drift from the codebase as work continues — treat any specific line number or exact field list as approximate once meaningful time has passed, and re-verify against the live source before relying on it for a consequential change.